밀키트 업체 공급망 관리를 위한 DB 구축

Team 09

2022147009 정호진
2020147005 김성민

Index.

01 - DB Requirement Analysis

02 - ER model

03 - CREATE TABLE
- INSERT DATA
- CREATE VIEW

04 - SQL/PSM
1. CREATE FUNCTION
2. CREATE TRIGGER
3. CREATE PROCEDURE

05 - Conclusions

1. DB Requirement Analysis

- 밀키트 수요는 1~2인 가구, 맞벌이 가구 중심으로 꾸준한 성장세를 보여왔음

- 코로나 19로 내식, 비대면 선호 소비자 증가

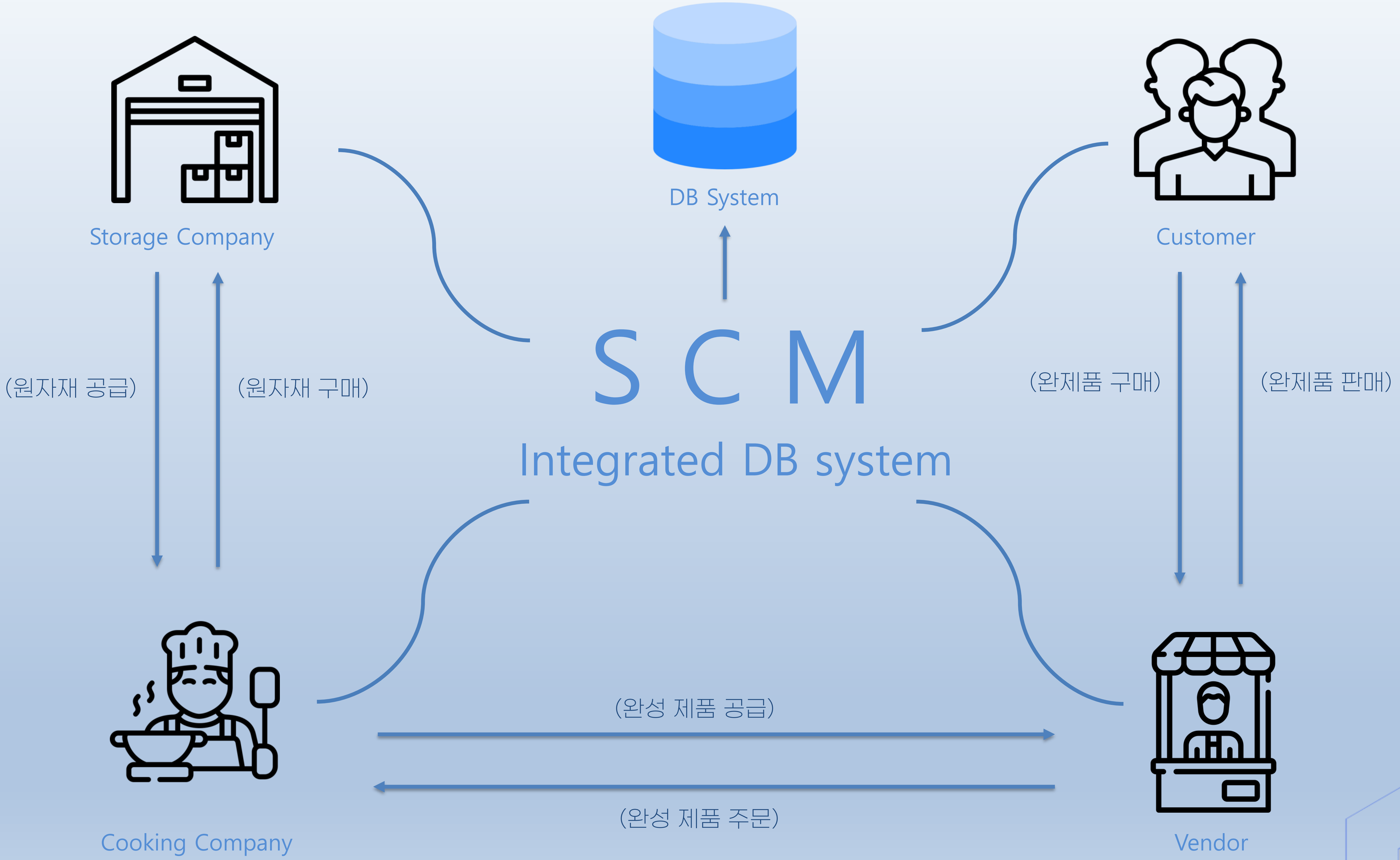
→ 밀키트 판매점의 완제품 주문 증가

→ 제조사: 제조물품 공급 및 원자재 조달 필요 (* 원자재는 냉동창고에 보관)

∴ 전체 과정이 공급망의 형태를 띔

→ 소매업체, 제조업체, 원자재 보관업체 각각의 운영 최적화를 위해
전체 공급망 구성원의 데이터를 관리하는 데이터베이스를 구축하여 프로세스의 효율성 추구

→ 각 구성원들의 최적의 의사결정을 돕고, 나아가 밀키트 산업의 동향성을 바라보는데 도움이 되기 위하여
이러한 데이터베이스를 구축하였음



SCM

Integrated DB system

DB System

Storage Company

Customer

(원자재 공급)

(원자재 구매)

(완제품 구매)

(완제품 판매)

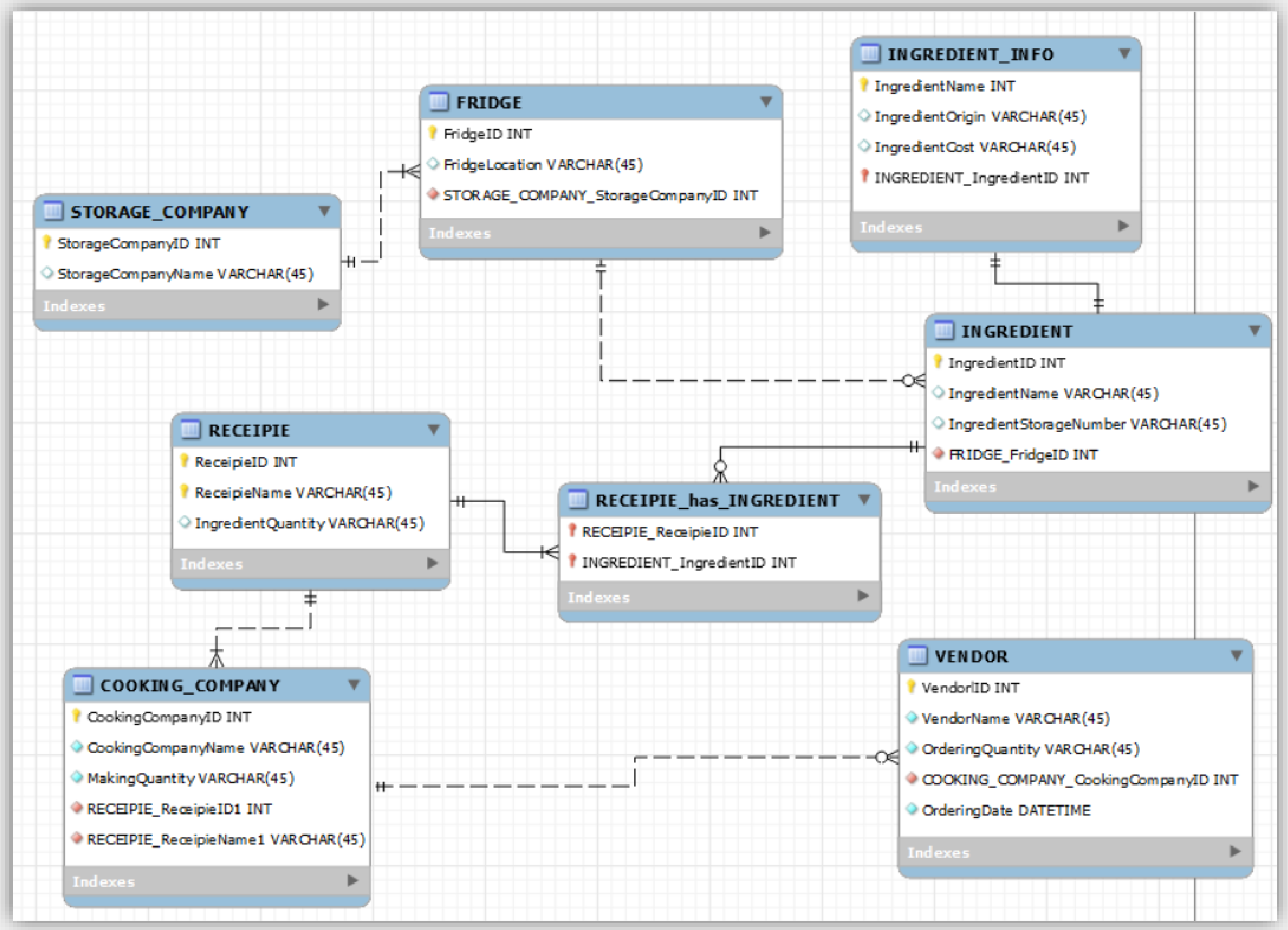
Cooking Company

Vendor

(완성 제품 공급)

(완성 제품 주문)

2. ER Modeling - Table


INGREDIENT_INFO
IngredientName INT
IngredientOrigin VARCHAR(45)
IngredientCost VARCHAR(45)
INGREDIENT_IngredientID INT
Indexes
FRIDGE
FridgeID INT
FridgeLocation VARCHAR(45)
STORAGE_COMPANY_StorageCompanyID INT
Indexes
STORAGE_COMPANY
StorageCompanyID INT
StorageCompanyName VARCHAR(45)
Indexes
INGREDIENT
IngredientID INT
IngredientName VARCHAR(45)
IngredientStorageNumber VARCHAR(45)
FRIDGE_FridgeID INT
Indexes
RECEIPIE
ReceipieID INT
ReceipieName VARCHAR(45)
IngredientQuantity VARCHAR(45)
Indexes
RECEIPIE_has_INGREDIENT
RECEIPIE_ReceipieID INT
INGREDIENT_IngredientID INT
Indexes
VENDOR
VendorID INT
VendorName VARCHAR(45)
OrderingQuantity VARCHAR(45)
COOKING_COMPANY_CookingCompanyID INT
OrderingDate DATETIME
Indexes
COOKING_COMPANY
CookingCompanyID INT
CookingCompanyName VARCHAR(45)
MakingQuantity VARCHAR(45)
RECEIPIE_ReceipieID1 INT
RECEIPIE_ReceipieName1 VARCHAR(45)
Indexes

STORAGE_COMPANY(StorageCompanyID, StorageCompanyName)

FRIDGE(FridgeID, FridgeLocation, *StorageCompanyID*)

INGREDIENT(IngredientID, IngredientName, … , *FridgeID*)

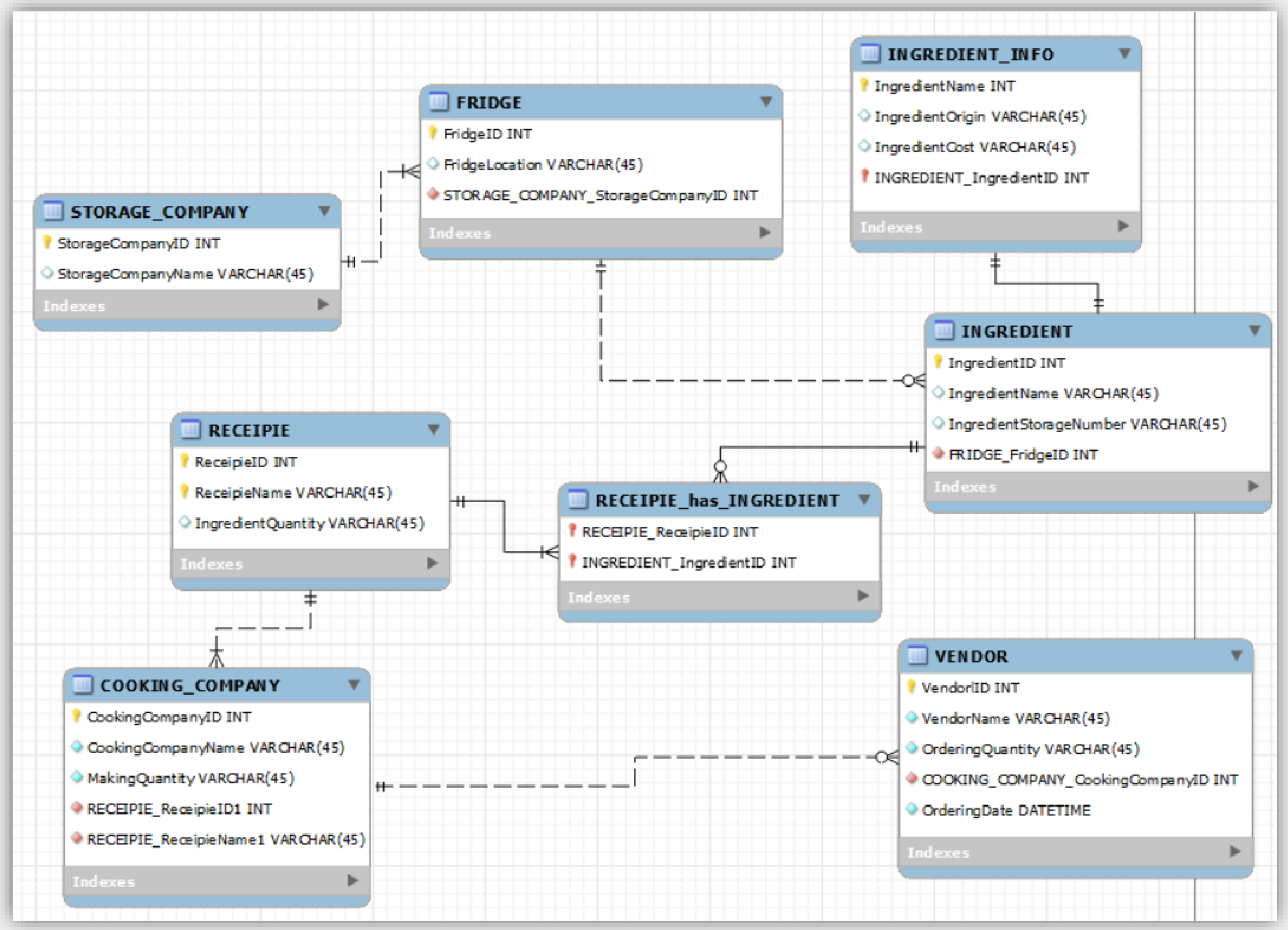
INGREDIENT_INFO(IngredientName, *IngredientID*, …)

RECEIPIE(ReceipieID, ReceipieName, IngredientQuantity)

COOKING_COMPANY(CookingCompanyID, … , *ReceipieID*, *ReceipieName*)

VENDOR(VendorID, VendorName, … , *CookingCompanyID*)

2. ER Modeling - Cardinality


STORAGE_COMPANY
StorageCompanyID INT
StorageCompanyName VARCHAR(45)
Indexes
FRIDGE
FridgeID INT
FridgeLocation VARCHAR(45)
STORAGE_COMPANY_StorageCompanyID INT
Indexes
INGREDIENT_INFO
IngredientName INT
IngredientOrigin VARCHAR(45)
IngredientCost VARCHAR(45)
INGREDIENT_IngredientID INT
Indexes
INGREDIENT
IngredientID INT
IngredientName VARCHAR(45)
IngredientStorageNumber VARCHAR(45)
FRIDGE_FridgeID INT
Indexes
RECEIPIE
ReceipieID INT
ReceipieName VARCHAR(45)
IngredientQuantity VARCHAR(45)
Indexes
RECEIPIE_has_INGREDIENT
RECEIPIE_ReceipieID INT
INGREDIENT_IngredientID INT
Indexes
COOKING_COMPANY
CookingCompanyID INT
CookingCompanyName VARCHAR(45)
MakingQuantity VARCHAR(45)
RECEIPIE_ReceipieID1 INT
RECEIPIE_ReceipieName1 VARCHAR(45)
Indexes
VENDOR
VendorID INT
VendorName VARCHAR(45)
OrderingQuantity VARCHAR(45)
COOKING_COMPANY_CookingCompanyID INT
OrderingDate DATETIME
Indexes

- 하나의 STORAGE_COMPANY는 여러 개의 FRIDGE를 가질 수 있으며, 최소 하나의 FRIDGE는 가져야 한다.
  한 개의 FRIDGE는 반드시 한 STORAGE_COMPANY의 소유이다.

- 하나의 FRIDGE에는 여러 종류의 INGREDIENT가 있을 수 있고, FRIDE에 INGREDIENT가 없어도 된다.
  한 INGREDIENT는 반드시 한 FRIDGE에 들어가 있어야 한다.

- INGREDIENT_INFO는 INGREDIENT의 부가 설명으로, INGREDIENT와 1:1, M:M 관계이며,
  INGREDIENT_INFO는 INGREDIENT에 ID-DEPENDENT하다.

- . INGREDIENT는 여러 RECEIPE에 포함될 수 있고, RECEIPE 또한 여러 INGREDIENT를 가질 수 있는 N:M 관계로, RECEIPE_has_INGREDIENT이 Intersection table이다.
  RECEIPE은 반드시 INGREDIENT를 가져야 하고, RECEIPE에 포함되지 않은 INGREDIENT가 있을 수 있다.

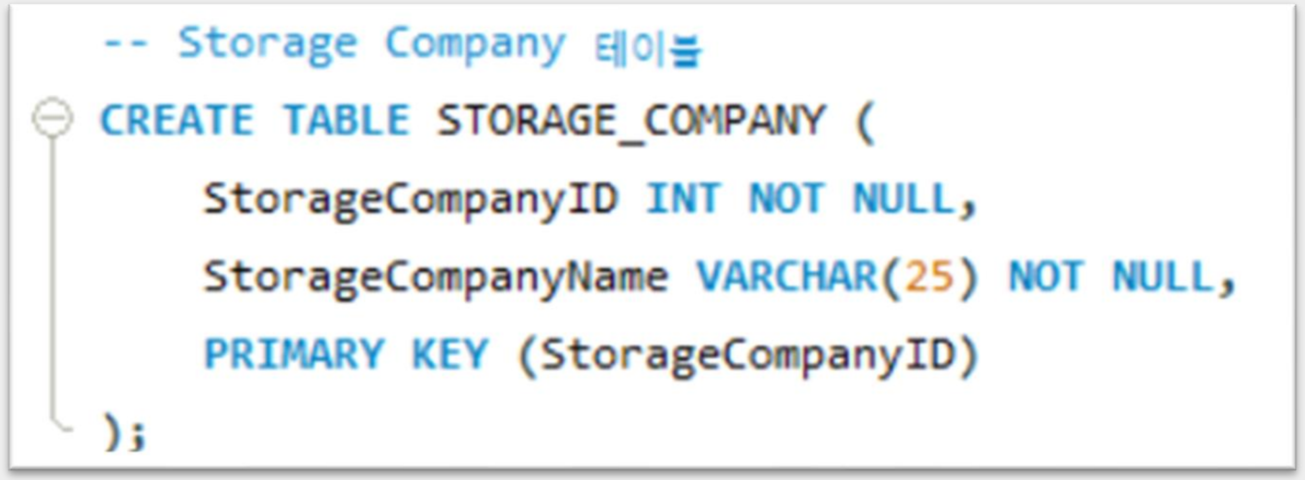
- 하나의 RECEIPE은 여러 COOKING_COMPANY에 포함될 수 있으며 반드시 하나의 COOKING_COMPANY에서 사용되어야 하고,
  한 COOKING_COMPANY에서는 반드시 하나의 RECEIPE만을 이용한다.

- COOKING_COMPANY는 여러 VENDOR에 상품을 공급할 수 있으며, 공급하지 않아도 된다.
  VENDOR은 반드시 하나의 COOKING_COMPANY에서만 상품을 공급 받는다.

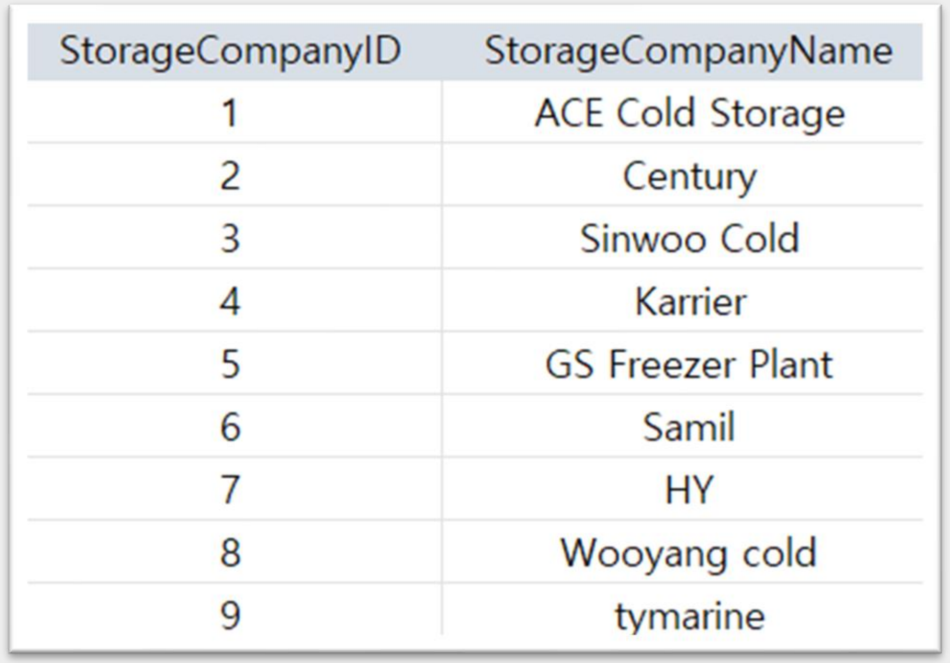
3. CREATE TABLE / INSERT DATA

1. STORAGE_COMPANY

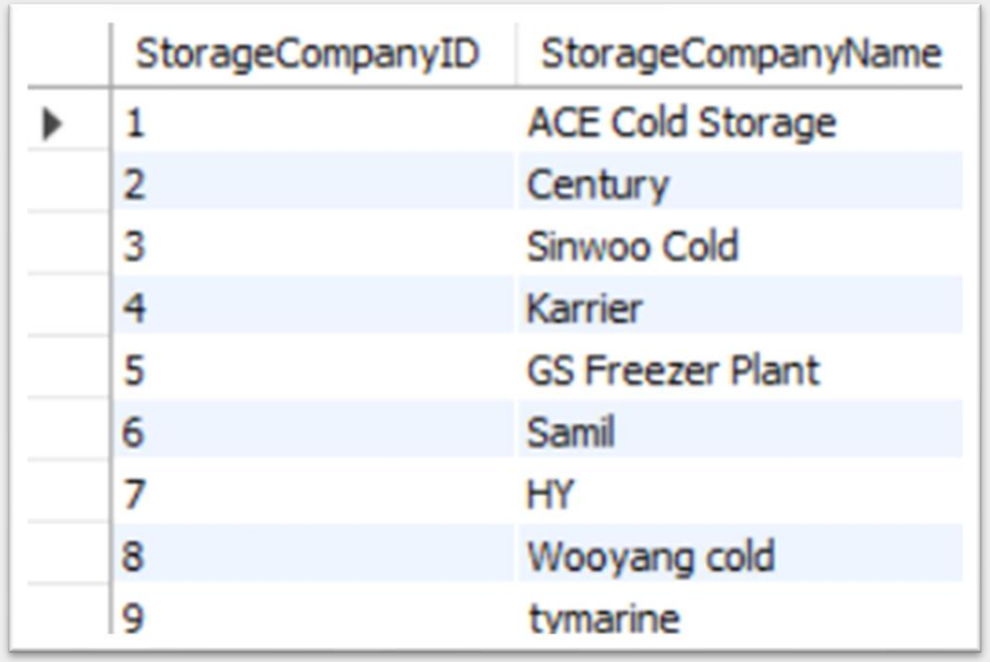
- 재료를 보관할 냉동고를 소유하는 업체에 대한 정보
- 업체 ID(StorageCompanyID)와
  업체 이름(StorageCompanyName)으로 구성
- Primary key: StorageCompanyID
- 실제 대형 냉동창고를 소유하는 회사를 찾아 CSV 파일 형태로 제작
  -> MySQL Data Import를 이용하여 Data Insert



```
-- Storage Company 테이블
CREATE TABLE STORAGE_COMPANY (
    StorageCompanyID INT NOT NULL,
    StorageCompanyName VARCHAR(25) NOT NULL,
    PRIMARY KEY (StorageCompanyID)
);
```



StorageCompanyID	StorageCompanyName
1	ACE Cold Storage
2	Century
3	Sinwoo Cold
4	Karrier
5	GS Freezer Plant
6	Samil
7	HY
8	Wooyang cold
9	tymarine



StorageCompanyID	StorageCompanyName
1	ACE Cold Storage
2	Century
3	Sinwoo Cold
4	Karrier
5	GS Freezer Plant
6	Samil
7	HY
8	Wooyang cold
9	tymarine

3. CREATE TABLE / INSERT DATA

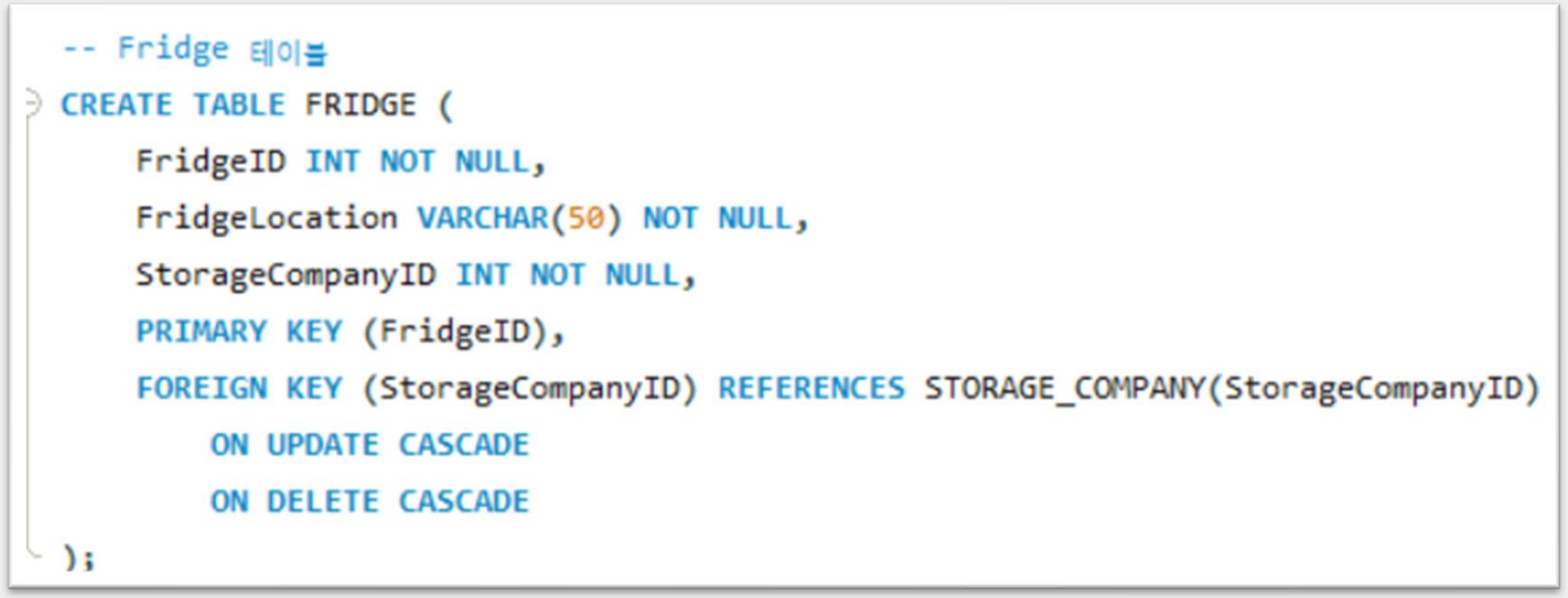
2. FRIDGE

- 재료를 실제로 보관하는 냉동고에 대한 정보

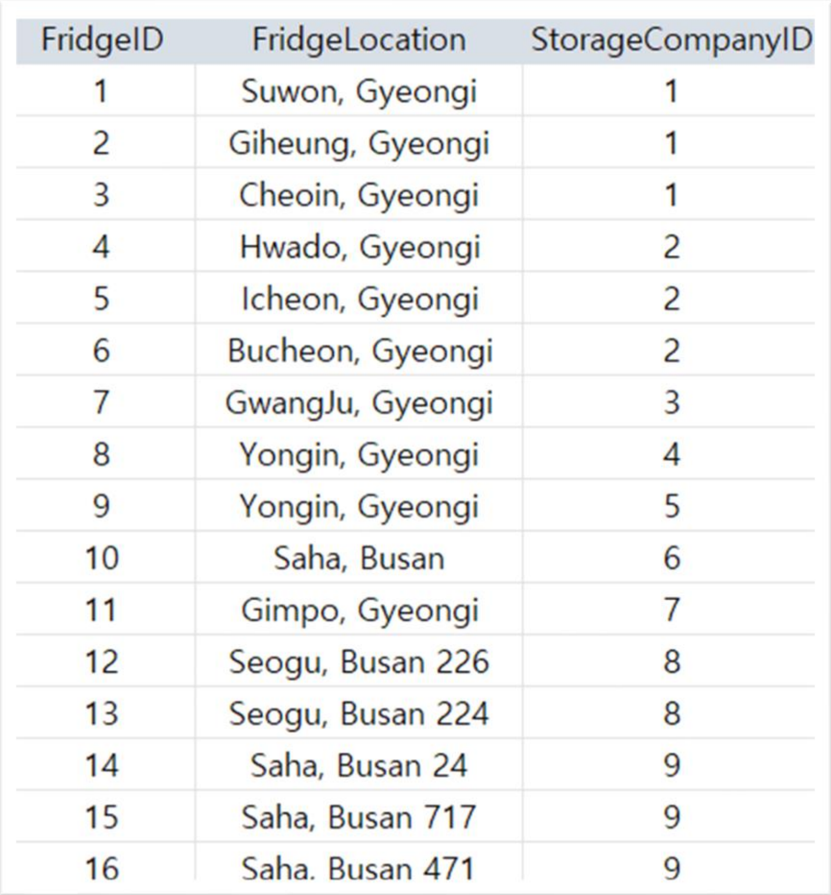
- 냉동고ID와 냉동고 위치로 구성
- STORAGE_COMPANY의 PK인 StorageCompanyID를 참조
- STORAGE_COMPANY와의 관계는
  CASCADE UPDATE와, DELETE를 허용

- Primary Key: FridgeID
  Foreign Key: StorageCompanyID

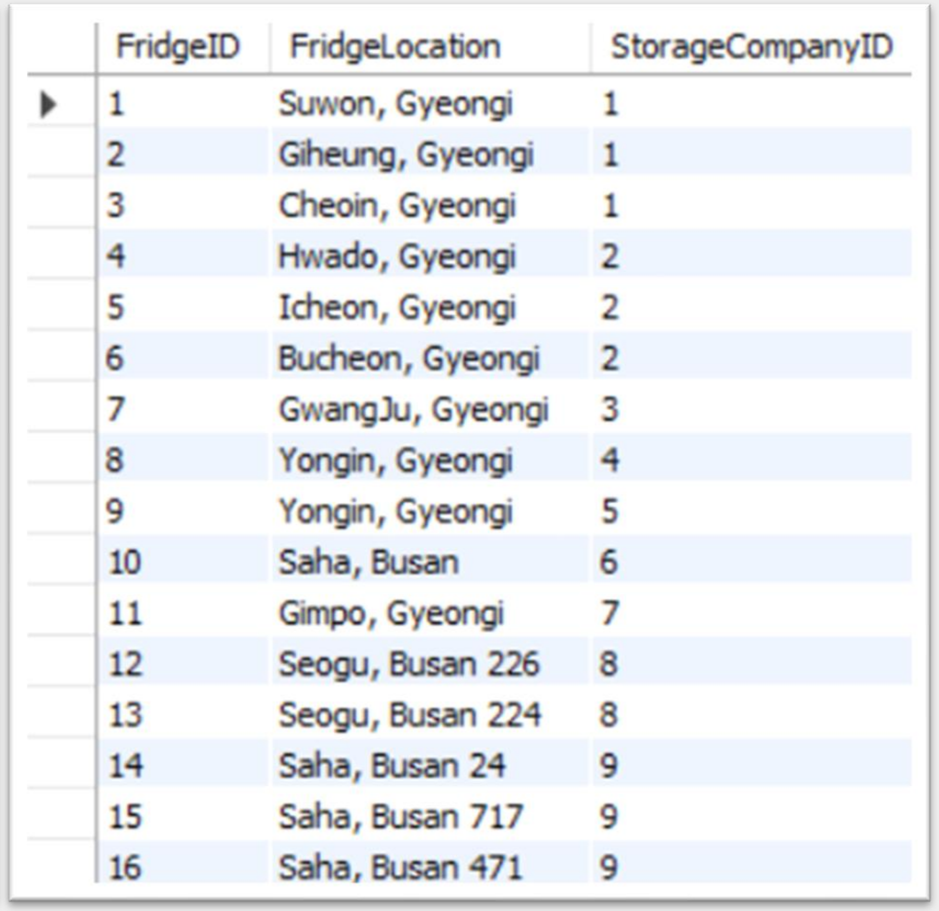
- STORAGE_COMPANY의 회사가 실제 소유하는 냉동고의 정보를 CSV 파일 형태로 제작
  -> MySQL Data Import를 이용하여 Data Insert



```
-- Fridge 테이블
CREATE TABLE FRIDGE (
    FridgeID INT NOT NULL,
    FridgeLocation VARCHAR(50) NOT NULL,
    StorageCompanyID INT NOT NULL,
    PRIMARY KEY (FridgeID),
    FOREIGN KEY (StorageCompanyID) REFERENCES STORAGE_COMPANY(StorageCompanyID)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);
```



FridgeID	FridgeLocation	StorageCompanyID
1	Suwon, Gyeongi	1
2	Giheung, Gyeongi	1
3	Cheoin, Gyeongi	1
4	Hwado, Gyeongi	2
5	Icheon, Gyeongi	2
6	Bucheon, Gyeongi	2
7	GwangJu, Gyeongi	3
8	Yongin, Gyeongi	4
9	Yongin, Gyeongi	5
10	Saha, Busan	6
11	Gimpo, Gyeongi	7
12	Seogu, Busan 226	8
13	Seogu, Busan 224	8
14	Saha, Busan 24	9
15	Saha, Busan 717	9
16	Saha, Busan 471	9



FridgeID	FridgeLocation	StorageCompanyID
1	Suwon, Gyeongi	1
2	Giheung, Gyeongi	1
3	Cheoin, Gyeongi	1
4	Hwado, Gyeongi	2
5	Icheon, Gyeongi	2
6	Bucheon, Gyeongi	2
7	GwangJu, Gyeongi	3
8	Yongin, Gyeongi	4
9	Yongin, Gyeongi	5
10	Saha, Busan	6
11	Gimpo, Gyeongi	7
12	Seogu, Busan 226	8
13	Seogu, Busan 224	8
14	Saha, Busan 24	9
15	Saha, Busan 717	9
16	Saha, Busan 471	9

3. CREATE TABLE / INSERT DATA

3. INGREDIENT

- 냉동고에 보관되는 재료

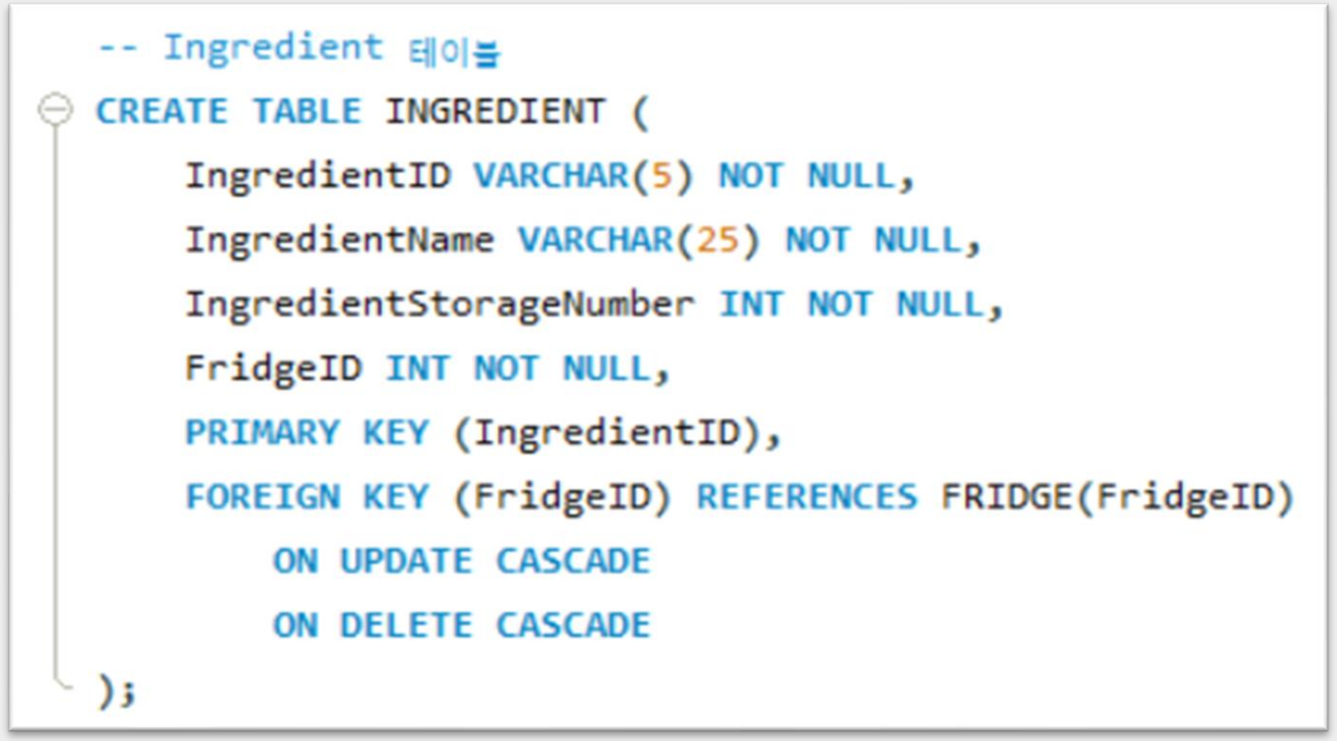
- 재료ID, 재료이름, 재료 보관 개수로 구성
  FRIDGE의 PK인 FridgeID를 참조

- FRIDGE와의 관계는 CASCADE UPDATE와, DELETE를 허용

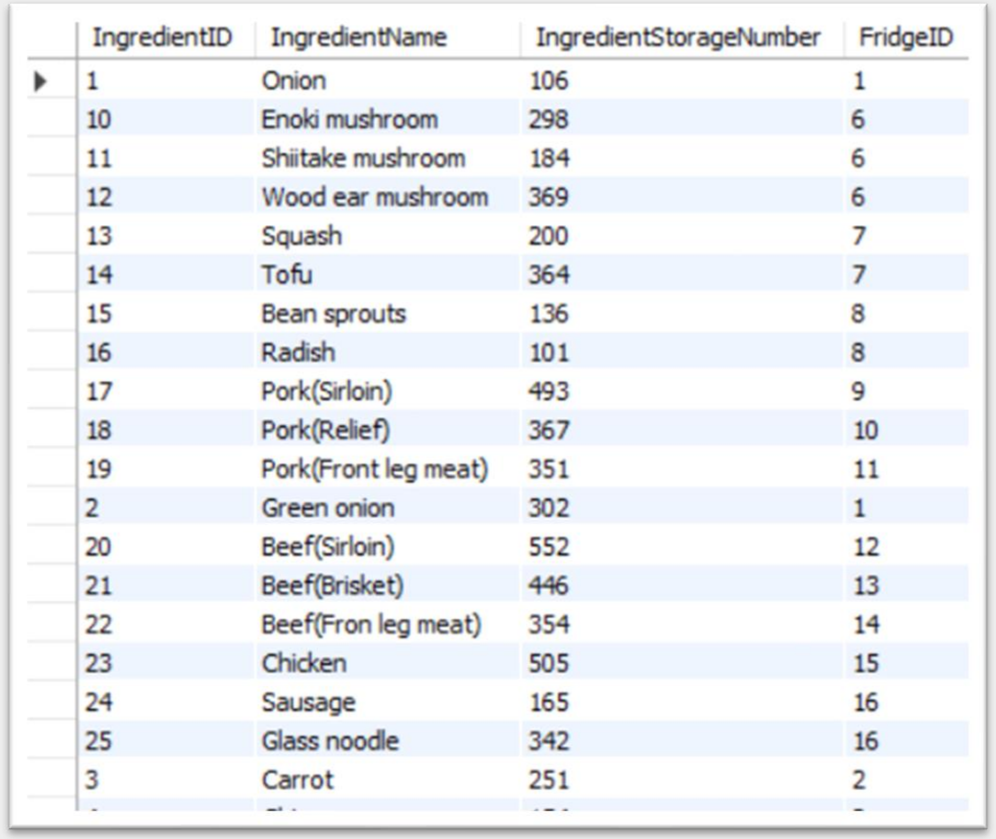
- Primary Key: IngredientID
  Foreign Key: FridgeID

- 밀키트에 사용될 요리를 정한 후 필요한 재료를 CSV형태로 제작
  -> MySQL에 Data Import를 통하여 입력

- 각 재료의 구체적인 수량을 알 수 없기에, Excel의 난수를 이용하여
  -> 밀키트를 제조하기에 충분한 수량으로 설정.



```sql
-- Ingredient 테이블
CREATE TABLE INGREDIENT (
    IngredientID VARCHAR(5) NOT NULL,
    IngredientName VARCHAR(25) NOT NULL,
    IngredientStorageNumber INT NOT NULL,
    FridgeID INT NOT NULL,
    PRIMARY KEY (IngredientID),
    FOREIGN KEY (FridgeID) REFERENCES FRIDGE(FridgeID)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);
```

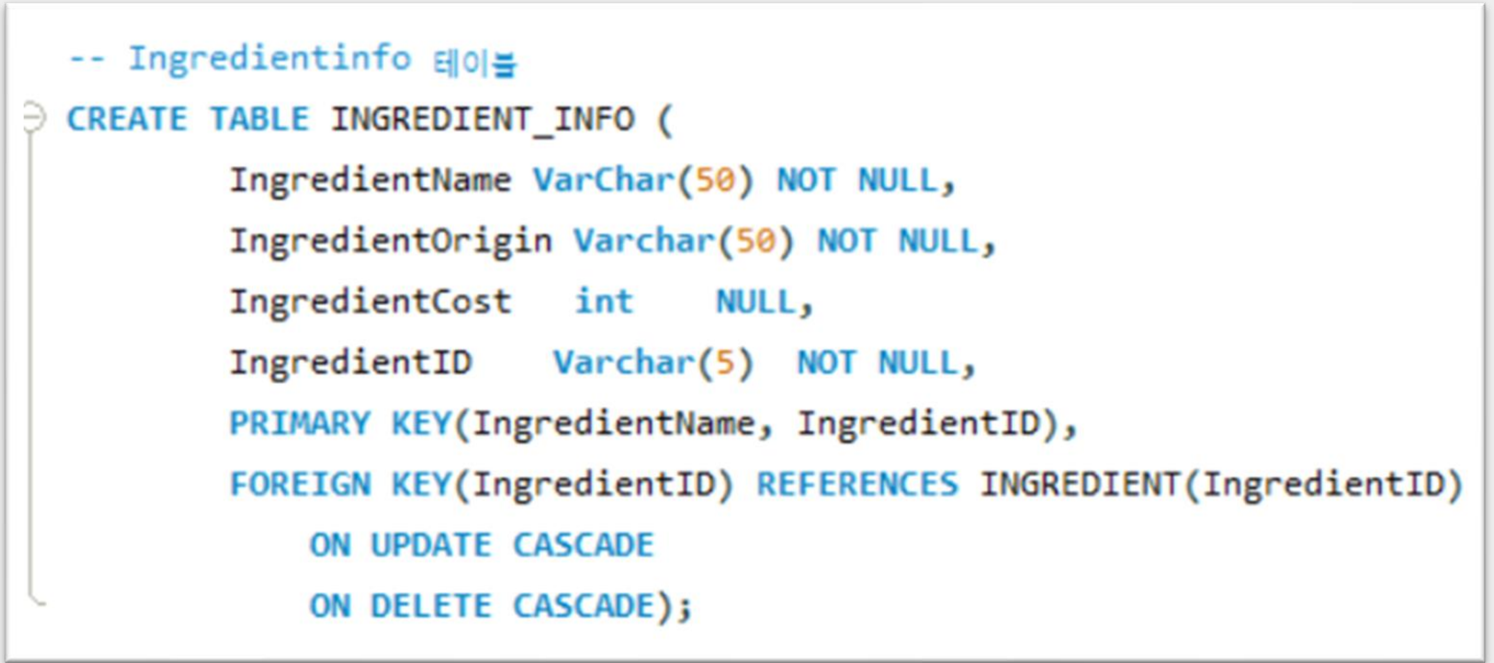


IngredientID	IngredientName	IngredientStorageNumber	FridgeID
1	Onion	106	1
10	Enoki mushroom	298	6
11	Shiitake mushroom	184	6
12	Wood ear mushroom	369	6
13	Squash	200	7
14	Tofu	364	7
15	Bean sprouts	136	8
16	Radish	101	8
17	Pork(Sirloin)	493	9
18	Pork(Relief)	367	10
19	Pork(Front leg meat)	351	11
2	Green onion	302	1
20	Beef(Sirloin)	552	12
21	Beef(Brisket)	446	13
22	Beef(Fron leg meat)	354	14
23	Chicken	505	15
24	Sausage	165	16
25	Glass noodle	342	16
3	Carrot	251	2

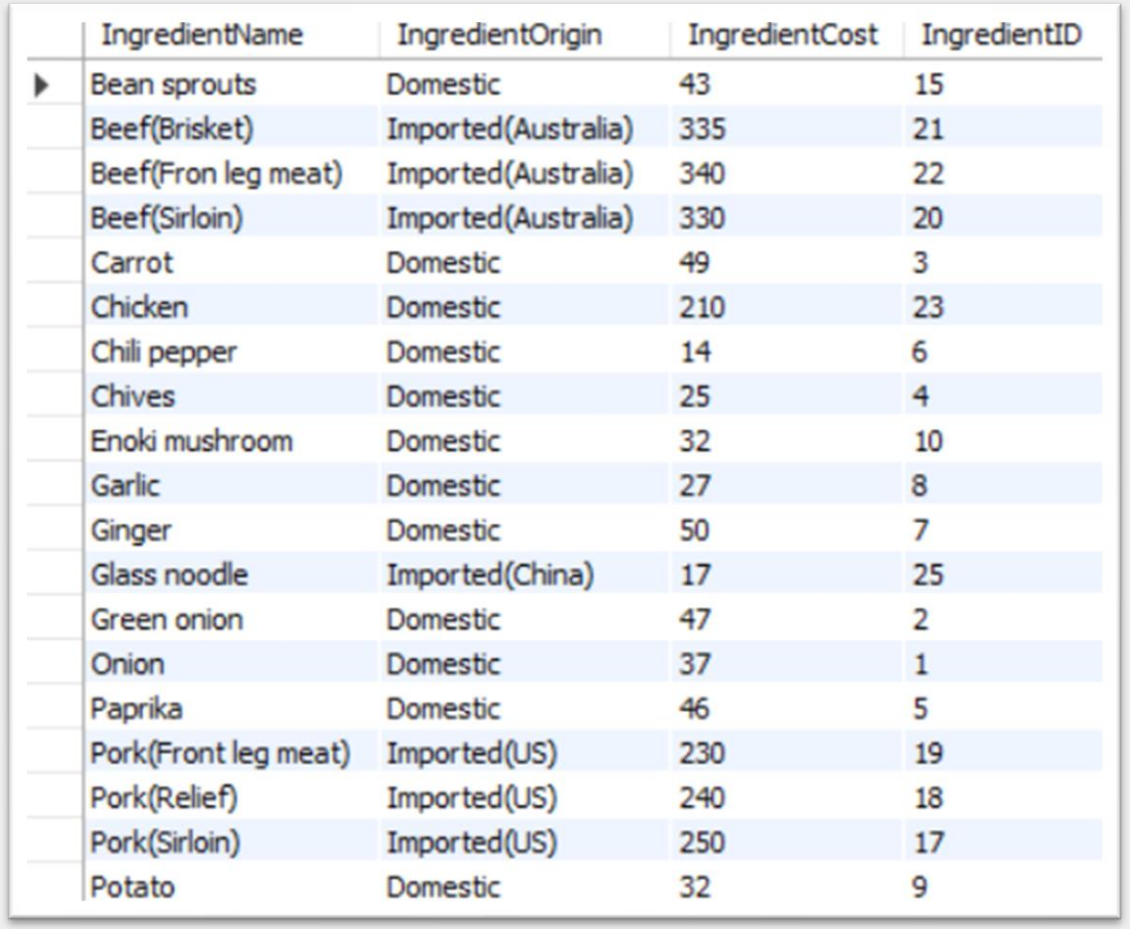
3. CREATE TABLE / INSERT DATA

3-1. INGREDIENT_INFO

- 냉동고에 보관되는 재료에 대한 정보
- 재료ID, 재료이름, 재료원산지, 단위당 재료 가격으로 구성
  INGREDIENT의 PK인 IngredientID를 참조
- INGREDIENT와의 관계는 CASCADE UPDATE와, DELETE를 허용
- Primary Key: IngredientID, IngredientName
  Foreign Key: IngredientID
- 재료의 정보를 CSV형태로 제작
  -> MySQL에 Data Import를 통하여 입력



```sql
-- Ingredientinfo 테이블
CREATE TABLE INGREDIENT_INFO (
        IngredientName VarChar(50) NOT NULL,
        IngredientOrigin Varchar(50) NOT NULL,
        IngredientCost   int    NULL,
        IngredientID    Varchar(5)  NOT NULL,
        PRIMARY KEY(IngredientName, IngredientID),
        FOREIGN KEY(IngredientID) REFERENCES INGREDIENT(IngredientID)
            ON UPDATE CASCADE
            ON DELETE CASCADE);
```



IngredientName	IngredientOrigin	IngredientCost	IngredientID
Bean sprouts	Domestic	43	15
Beef(Brisket)	Imported(Australia)	335	21
Beef(Fron leg meat)	Imported(Australia)	340	22
Beef(Sirloin)	Imported(Australia)	330	20
Carrot	Domestic	49	3
Chicken	Domestic	210	23
Chili pepper	Domestic	14	6
Chives	Domestic	25	4
Enoki mushroom	Domestic	32	10
Garlic	Domestic	27	8
Ginger	Domestic	50	7
Glass noodle	Imported(China)	17	25
Green onion	Domestic	47	2
Onion	Domestic	37	1
Paprika	Domestic	46	5
Pork(Front leg meat)	Imported(US)	230	19
Pork(Relief)	Imported(US)	240	18
Pork(Sirloin)	Imported(US)	250	17
Potato	Domestic	32	9

3. CREATE TABLE / INSERT DATA

4. COOKING_COMPANY

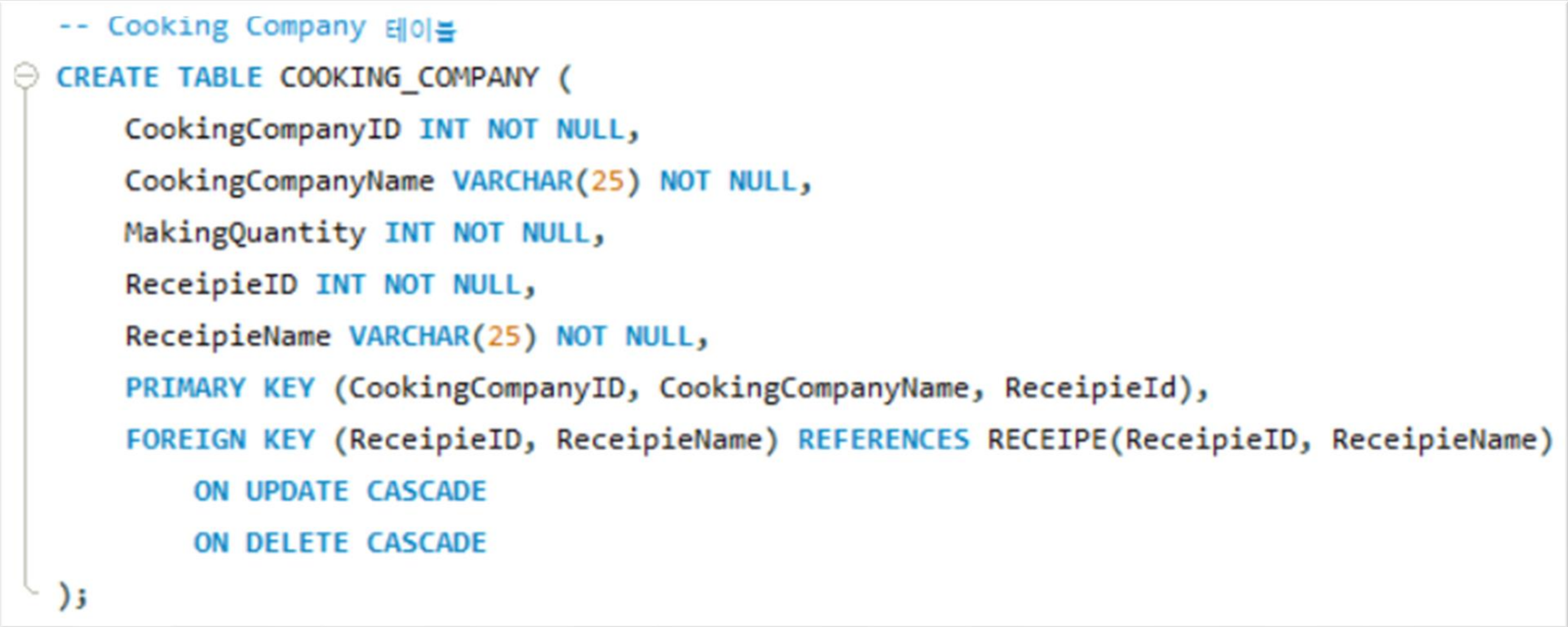
- 재료를 이용하여 밀키트 완성본을 제조하는 회사

- 회사ID, 회사 이름, 제조수량으로 구성
  RECEIPE의 PK인 ReceipeID와 ReceipeName을 참조

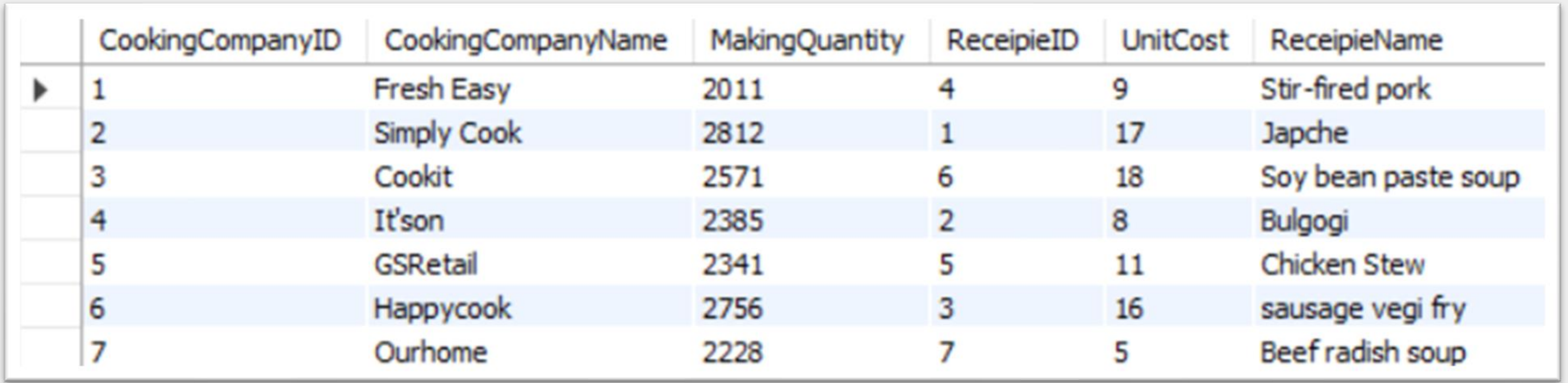
- RECEIPE 와의 관계는 CASCADE UPDATE와, DELETE를 허용

- Primary Key: CookingCompanyID, CookingCompanyName, ReceipieID
  Foreign Key: ReceipeID, ReceipeName

- 제조회사의 정보를 CSV형태로 제작
  -> MySQL에 Data Import를 통하여 입력



```sql
-- Cooking Company 테이블
CREATE TABLE COOKING_COMPANY (
    CookingCompanyID INT NOT NULL,
    CookingCompanyName VARCHAR(25) NOT NULL,
    MakingQuantity INT NOT NULL,
    ReceipieID INT NOT NULL,
    ReceipieName VARCHAR(25) NOT NULL,
    PRIMARY KEY (CookingCompanyID, CookingCompanyName, ReceipieId),
    FOREIGN KEY (ReceipieID, ReceipieName) REFERENCES RECEIPE(ReceipieID, ReceipieName)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);
```

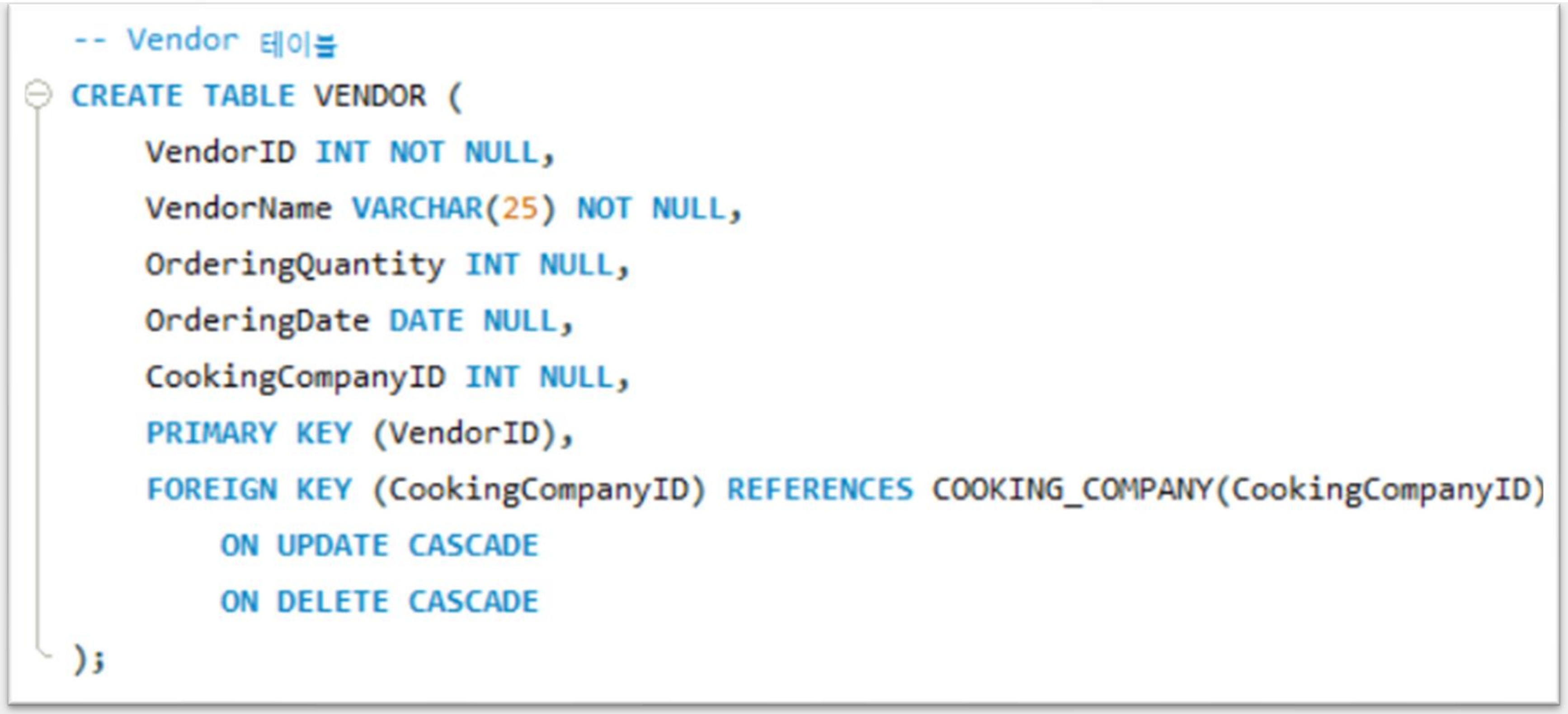


CookingCompanyID	CookingCompanyName	MakingQuantity	ReceipieID	UnitCost	ReceipieName
1	Fresh Easy	2011	4	9	Stir-fired pork
2	Simply Cook	2812	1	17	Japche
3	Cookit	2571	6	18	Soy bean paste soup
4	It'son	2385	2	8	Bulgogi
5	GSRetail	2341	5	11	Chicken Stew
6	Happycook	2756	3	16	sausage vegi fry
7	Ourhome	2228	7	5	Beef radish soup

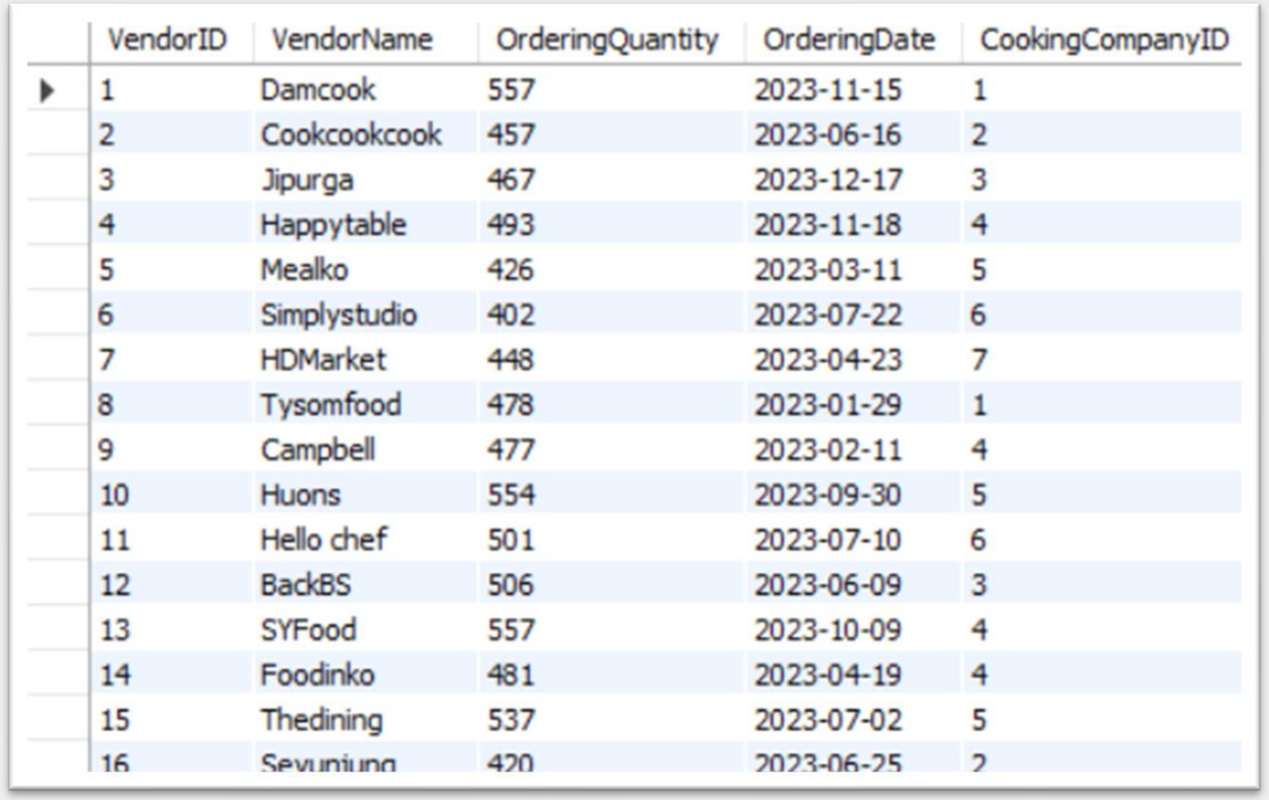
3. CREATE TABLE / INSERT DATA

5. VENDOR

- 제조회사로부터 완성된 밀키트를 수주하여 판매하는 판매점
- 판매점ID, 판매점 이름, 주문수량, 주문일자로 구성
  COOKING_COMPANY의 PK인 CookingCompanyID를 참조
- COOKING_COMPANY 와의 관계는 CASCADE UPDATE와 DELETE를 허용
- Primary Key: VendorID
  Foreign Key: CookingCompanyID
- 판매점의 정보를 CSV형태로 제작 후
  -> MySQL에 Data Import를 통하여 입력



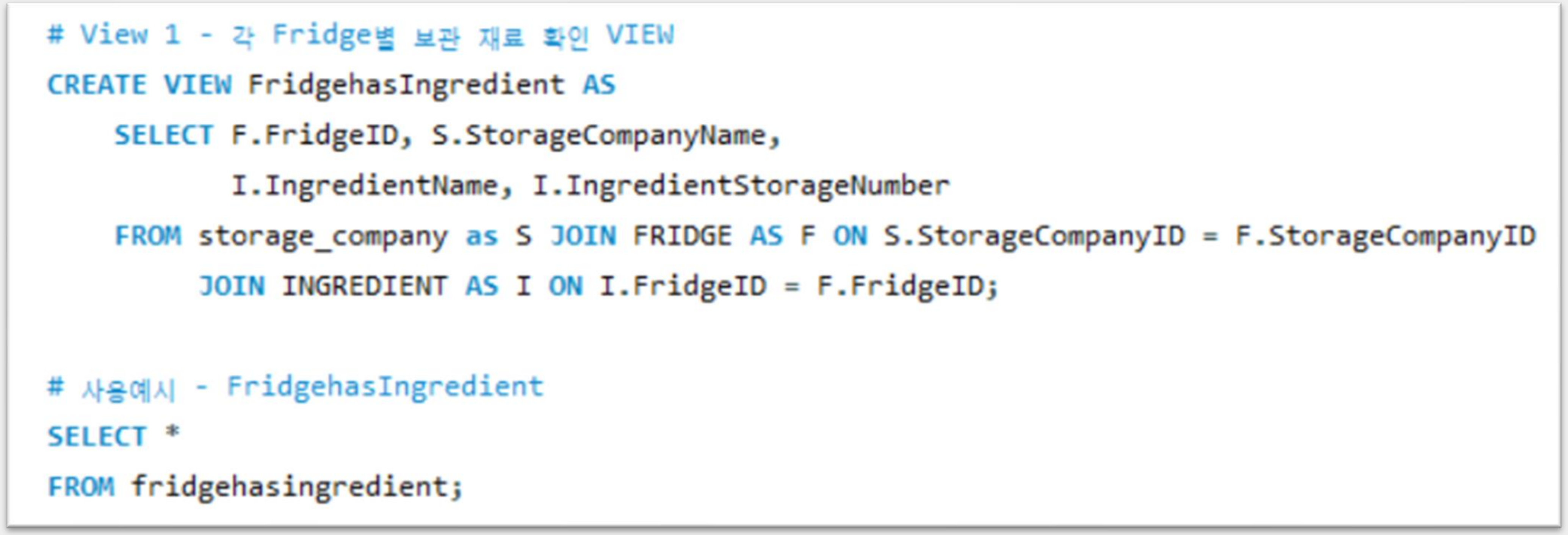
```
-- Vendor 테이블
CREATE TABLE VENDOR (
    VendorID INT NOT NULL,
    VendorName VARCHAR(25) NOT NULL,
    OrderingQuantity INT NULL,
    OrderingDate DATE NULL,
    CookingCompanyID INT NULL,
    PRIMARY KEY (VendorID),
    FOREIGN KEY (CookingCompanyID) REFERENCES COOKING_COMPANY(CookingCompanyID)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);
```



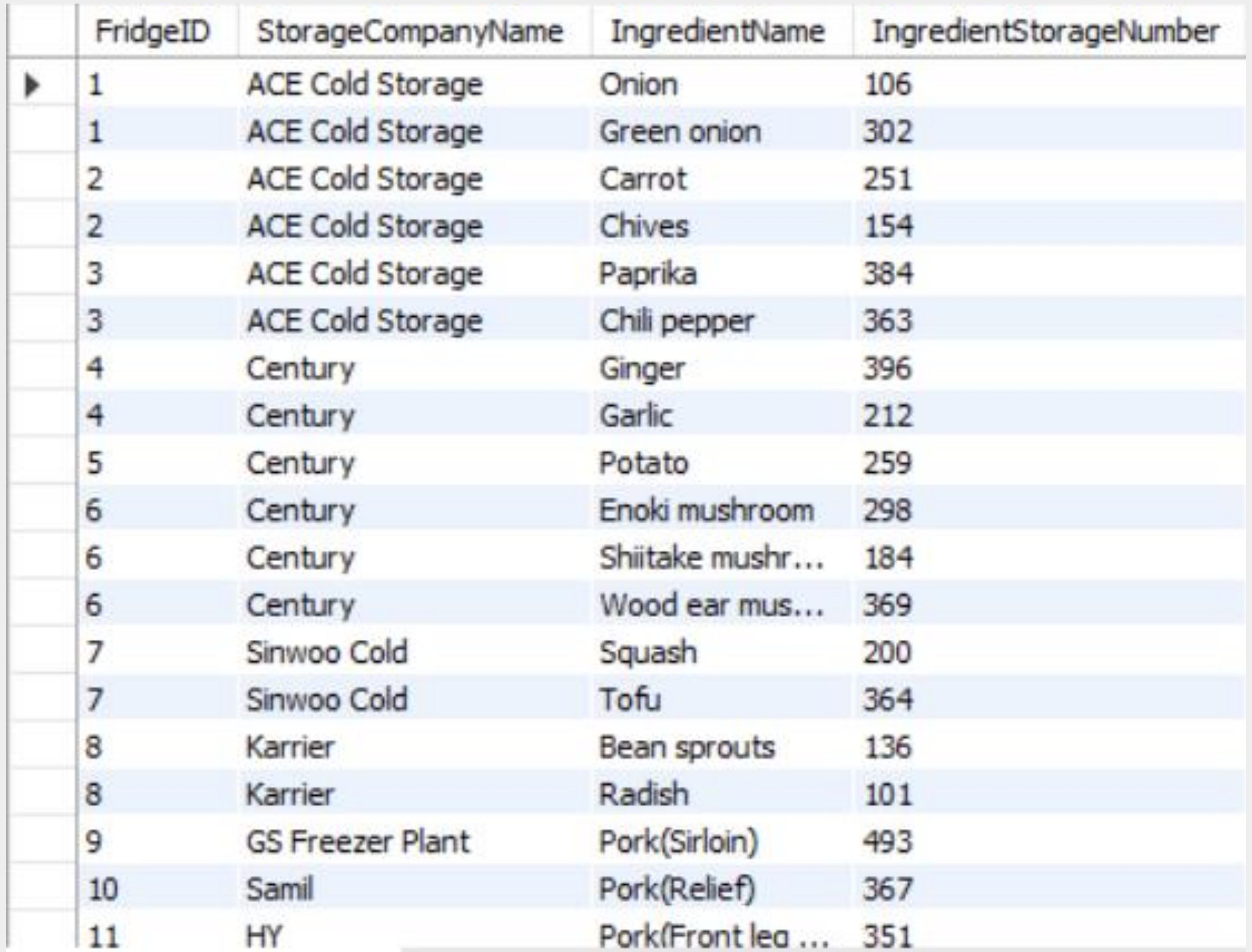
VendorID	VendorName	OrderingQuantity	OrderingDate	CookingCompanyID
1	Damcook	557	2023-11-15	1
2	Cookcookcook	457	2023-06-16	2
3	Jipurga	467	2023-12-17	3
4	Happytable	493	2023-11-18	4
5	Mealko	426	2023-03-11	5
6	Simplystudio	402	2023-07-22	6
7	HDMarket	448	2023-04-23	7
8	Tysomfood	478	2023-01-29	1
9	Campbell	477	2023-02-11	4
10	Huons	554	2023-09-30	5
11	Hello chef	501	2023-07-10	6
12	BackBS	506	2023-06-09	3
13	SYFood	557	2023-10-09	4
14	Foodinko	481	2023-04-19	4
15	Thedining	537	2023-07-02	5
16	Sevunjung	420	2023-06-25	2

3. CREATE VIEW - Fridge has Ingredient

1. 각 Fridge별 보관하고 있는 재료 총 개수의 확인



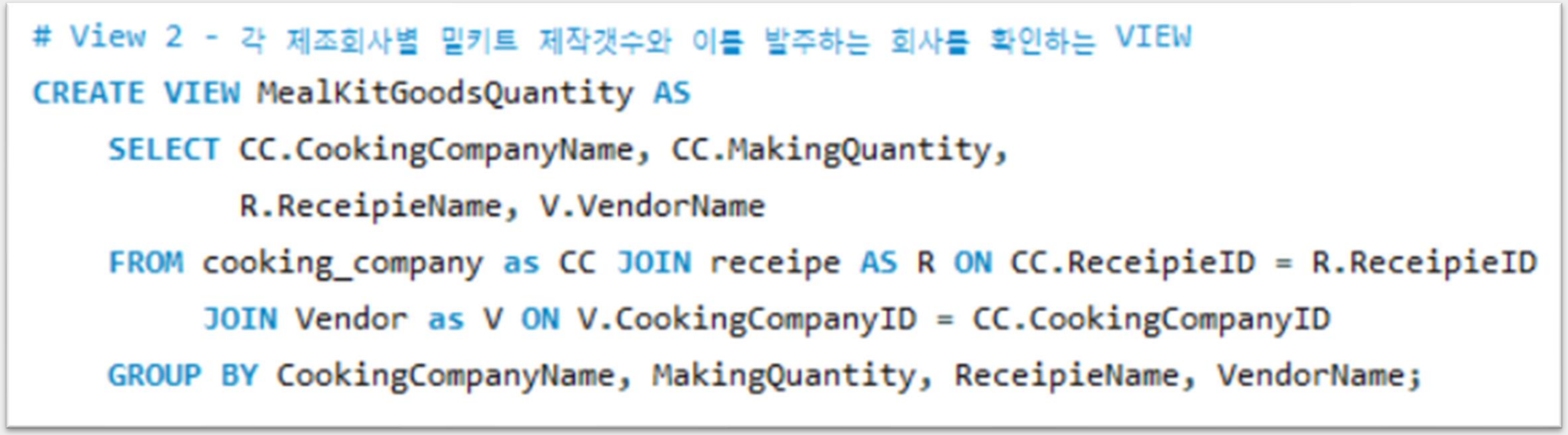
```
# View 1 - 각 Fridge별 보관 재료 확인 VIEW
CREATE VIEW FridgehasIngredient AS
    SELECT F.FridgeID, S.StorageCompanyName,
           I.IngredientName, I.IngredientStorageNumber
    FROM storage_company as S JOIN FRIDGE AS F ON S.StorageCompanyID = F.StorageCompanyID
         JOIN INGREDIENT AS I ON I.FridgeID = F.FridgeID;

# 사용예시 - FridgehasIngredient
SELECT *
FROM fridgehasingredient;
```



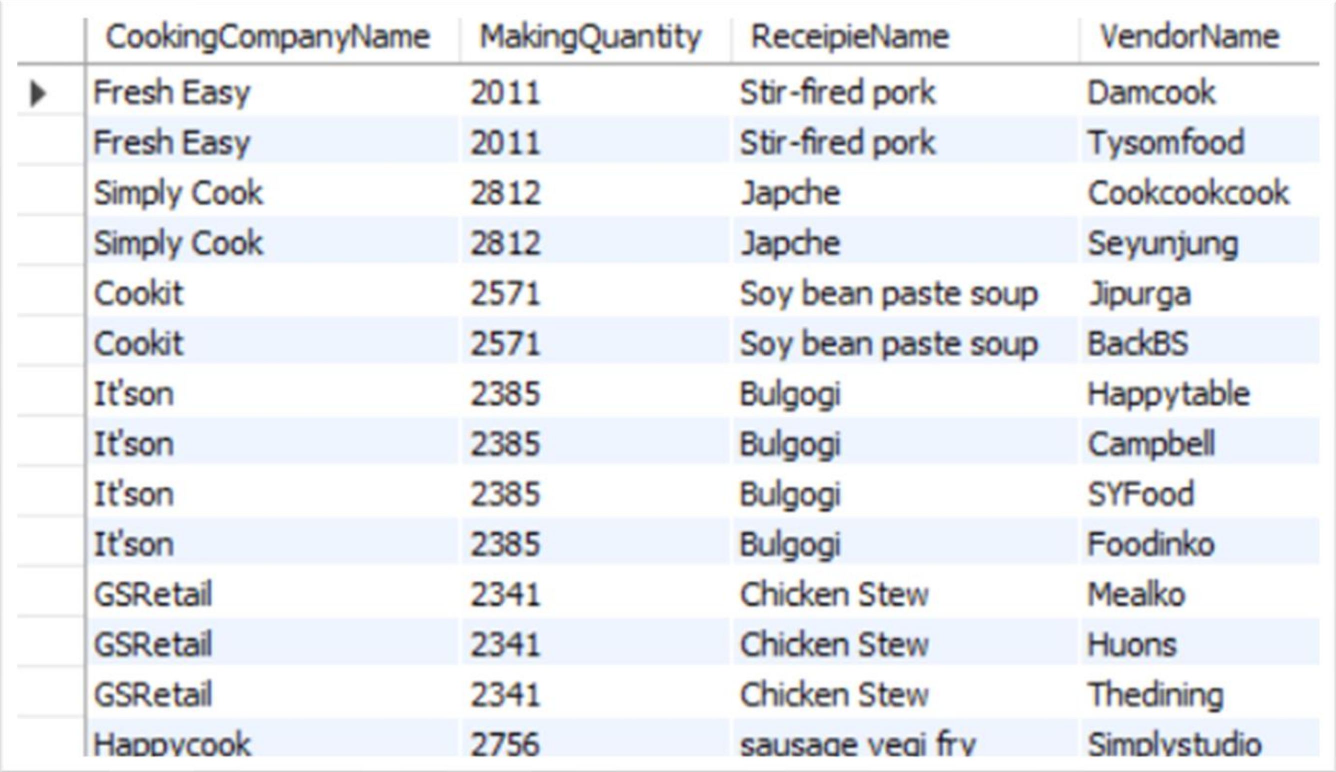
FridgeID	StorageCompanyName	IngredientName	IngredientStorageNumber
1	ACE Cold Storage	Onion	106
1	ACE Cold Storage	Green onion	302
2	ACE Cold Storage	Carrot	251
2	ACE Cold Storage	Chives	154
3	ACE Cold Storage	Paprika	384
3	ACE Cold Storage	Chili pepper	363
4	Century	Ginger	396
4	Century	Garlic	212
5	Century	Potato	259
6	Century	Enoki mushroom	298
6	Century	Shiitake mushr...	184
6	Century	Wood ear mus...	369
7	Sinwoo Cold	Squash	200
7	Sinwoo Cold	Tofu	364
8	Karrier	Bean sprouts	136
8	Karrier	Radish	101
9	GS Freezer Plant	Pork(Sirloin)	493
10	Samil	Pork(Relief)	367
11	HY	Pork(Front leg ...	351

3. CREATE VIEW - MealKit Goods Quantity

2. 각 제조회사별 밀키트 제작 개수와 이를 발주하는 회사를 확인



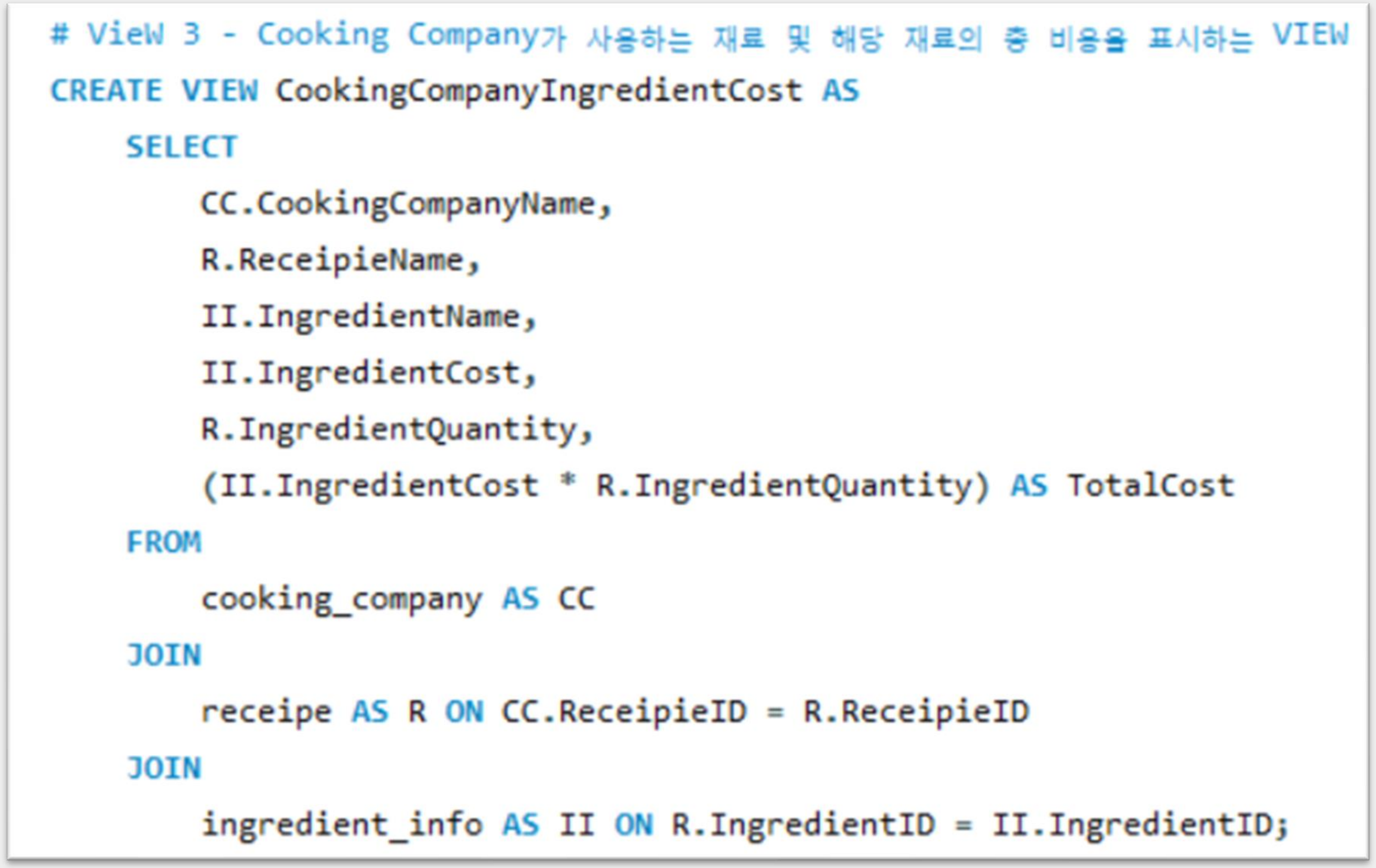
```
# View 2 - 각 제조회사별 밀키트 제작갯수와 이를 발주하는 회사를 확인하는 VIEW
CREATE VIEW MealKitGoodsQuantity AS
    SELECT CC.CookingCompanyName, CC.MakingQuantity,
           R.ReceipieName, V.VendorName
    FROM cooking_company as CC JOIN receipe AS R ON CC.ReceipieID = R.ReceipieID
         JOIN Vendor as V ON V.CookingCompanyID = CC.CookingCompanyID
    GROUP BY CookingCompanyName, MakingQuantity, ReceipieName, VendorName;
```



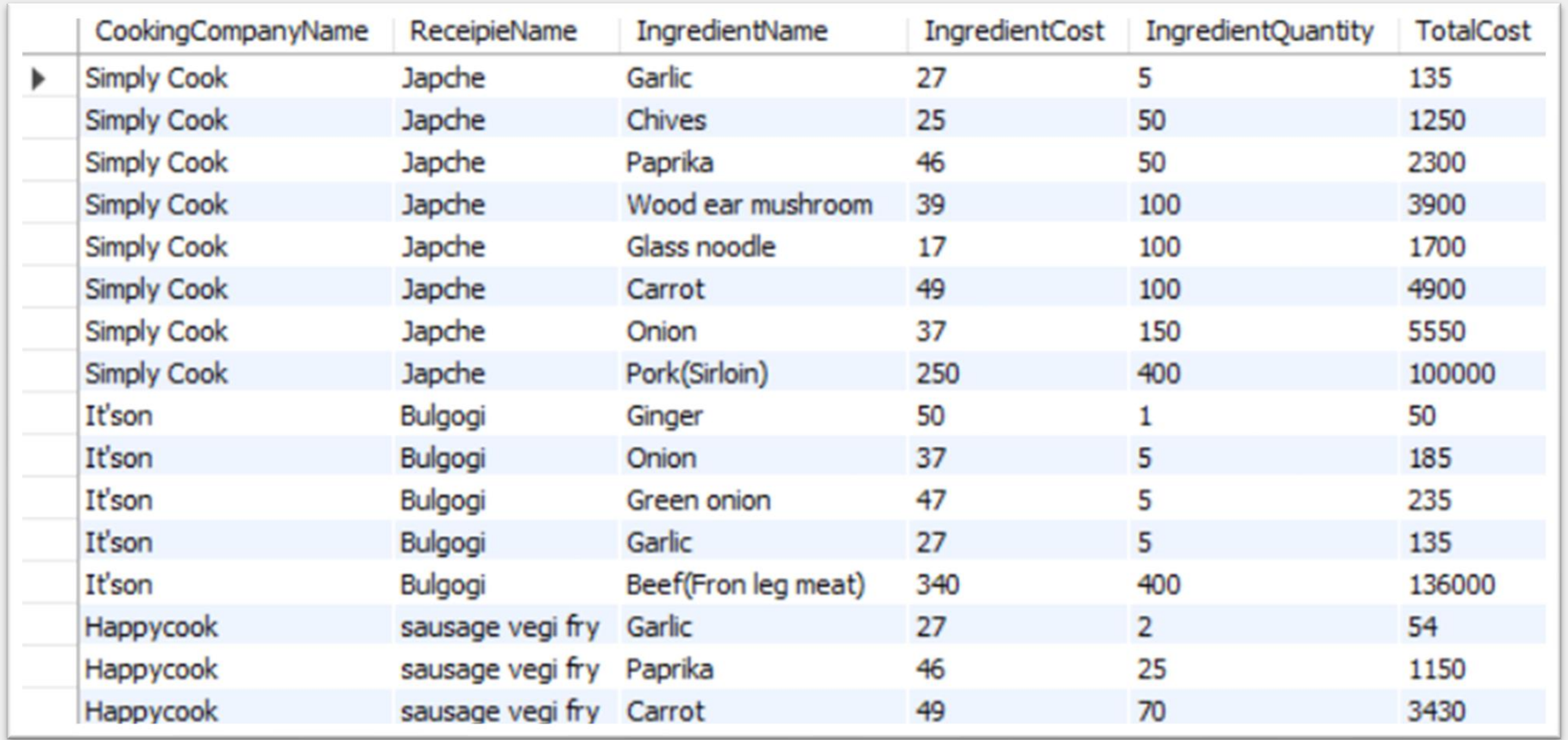
CookingCompanyName	MakingQuantity	ReceipieName	VendorName
Fresh Easy	2011	Stir-fired pork	Damcook
Fresh Easy	2011	Stir-fired pork	Tysomfood
Simply Cook	2812	Japche	Cookcookcook
Simply Cook	2812	Japche	Seyunjung
Cookit	2571	Soy bean paste soup	Jipurga
Cookit	2571	Soy bean paste soup	BackBS
It'son	2385	Bulgogi	Happytable
It'son	2385	Bulgogi	Campbell
It'son	2385	Bulgogi	SYFood
It'son	2385	Bulgogi	Foodinko
GSRetail	2341	Chicken Stew	Mealko
GSRetail	2341	Chicken Stew	Huons
GSRetail	2341	Chicken Stew	Thedining
Happycook	2756	sausage vegi fry	Simplystudio

3. CREATE VIEW - CookingCompany IngredientCost

3. 각 제조회사별 사용재료 및 해당 재료의 총 비용을 표시



```
# VieW 3 - Cooking Company가 사용하는 재료 및 해당 재료의 총 비용을 표시하는 VIEW
CREATE VIEW CookingCompanyIngredientCost AS
    SELECT
        CC.CookingCompanyName,
        R.ReceipieName,
        II.IngredientName,
        II.IngredientCost,
        R.IngredientQuantity,
        (II.IngredientCost * R.IngredientQuantity) AS TotalCost
    FROM
        cooking_company AS CC
    JOIN
        receipe AS R ON CC.ReceipieID = R.ReceipieID
    JOIN
        ingredient_info AS II ON R.IngredientID = II.IngredientID;
```



	CookingCompanyName	ReceipieName	IngredientName	IngredientCost	IngredientQuantity	TotalCost
▶	Simply Cook	Japche	Garlic	27	5	135
	Simply Cook	Japche	Chives	25	50	1250
	Simply Cook	Japche	Paprika	46	50	2300
	Simply Cook	Japche	Wood ear mushroom	39	100	3900
	Simply Cook	Japche	Glass noodle	17	100	1700
	Simply Cook	Japche	Carrot	49	100	4900
	Simply Cook	Japche	Onion	37	150	5550
	Simply Cook	Japche	Pork(Sirloin)	250	400	100000
	It'son	Bulgogi	Ginger	50	1	50
	It'son	Bulgogi	Onion	37	5	185
	It'son	Bulgogi	Green onion	47	5	235
	It'son	Bulgogi	Garlic	27	5	135
	It'son	Bulgogi	Beef(Fron leg meat)	340	400	136000
	Happycook	sausage vegi fry	Garlic	27	2	54
	Happycook	sausage vegi fry	Paprika	46	25	1150
	Happycook	sausage vegi fry	Carrot	49	70	3430

4. SQL/PSM - Trigger

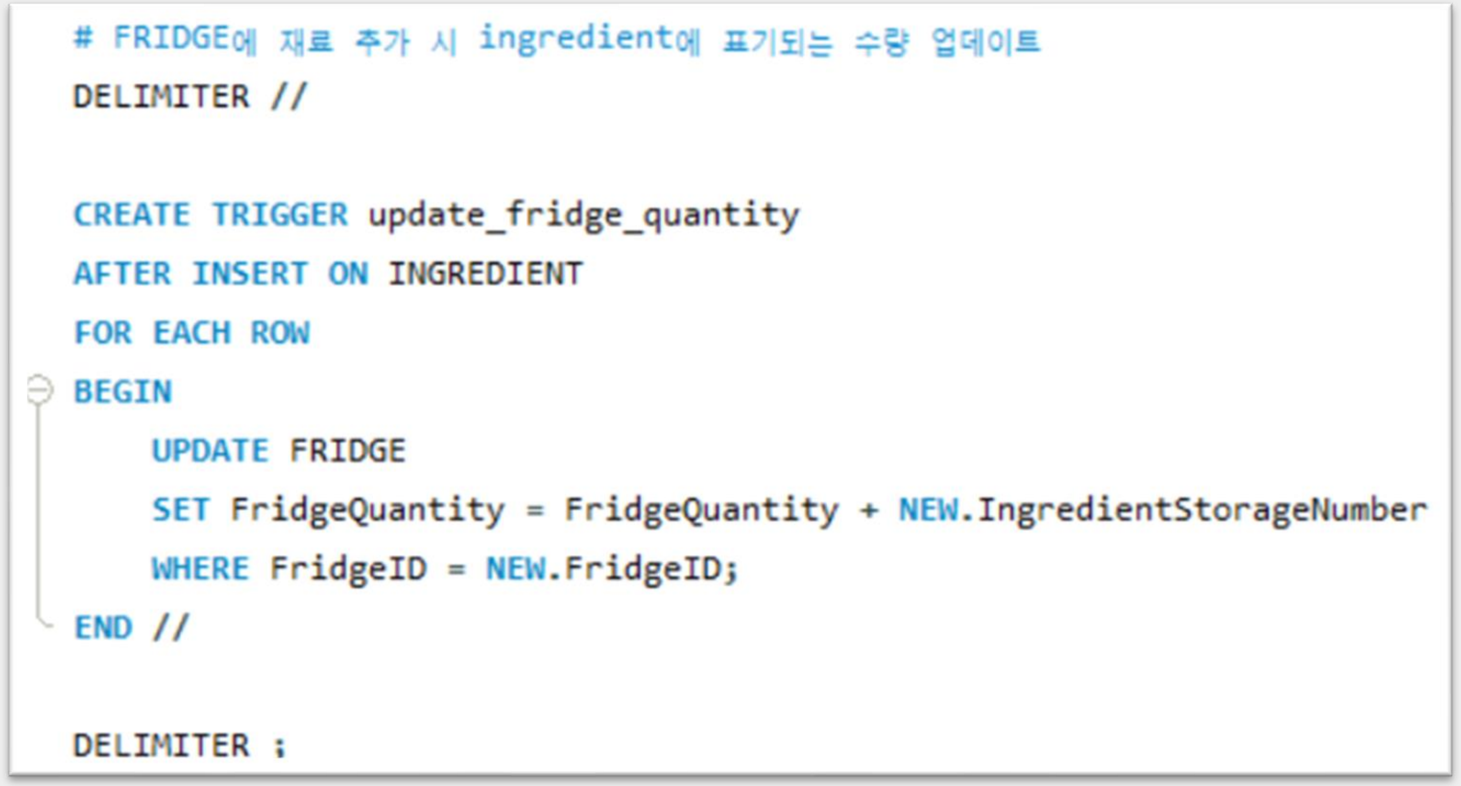
1. Trigger

(1) update_fridge_quantity: 냉동고에 재료 추가 시 INGREDIENT에 표기되는 재료 수량 업데이트

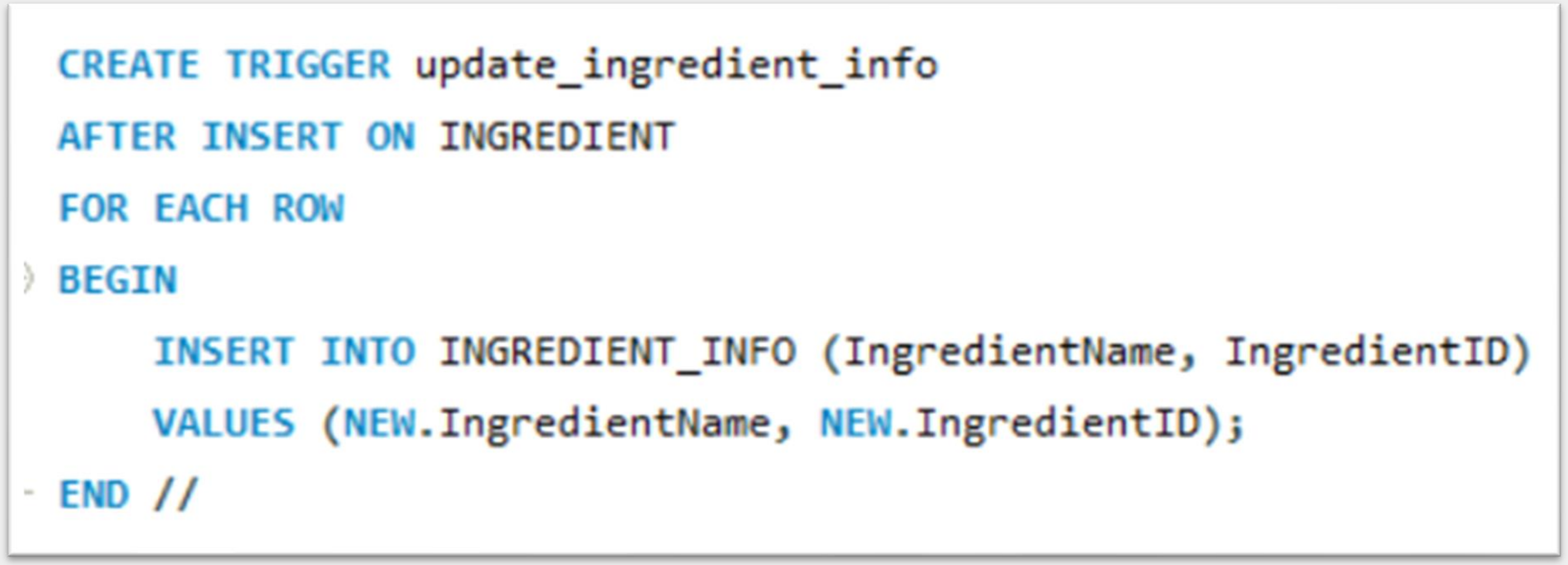
- 재료 수량 변동 시 밀키트 생산용량에 변화 가능

(2) update_ingredient_info: 새로운 INGREDIENT가 추가될 시 INGREDIENT_INFO에 자동으로 재료정보 등록

- 새로운 INGREDIENT가 등록될 시, 새로운 밀키트 제품을 생산할 수 있기에 바로 관련정보를 등록
- 이러한 Action이 INGREDINT, INGREDIENT_INFO에 정보 등록 이후 발생하기에, AFTER로 Trigger를 설정



```sql
# FRIDGE에 재료 추가 시 ingredient에 표기되는 수량 업데이트
DELIMITER //

CREATE TRIGGER update_fridge_quantity
AFTER INSERT ON INGREDIENT
FOR EACH ROW
BEGIN
    UPDATE FRIDGE
    SET FridgeQuantity = FridgeQuantity + NEW.IngredientStorageNumber
    WHERE FridgeID = NEW.FridgeID;
END //

DELIMITER ;
```



```sql
CREATE TRIGGER update_ingredient_info
AFTER INSERT ON INGREDIENT
FOR EACH ROW
BEGIN
    INSERT INTO INGREDIENT_INFO (IngredientName, IngredientID)
    VALUES (NEW.IngredientName, NEW.IngredientID);
END //
```

4. SQL/PSM - Stored Procedure

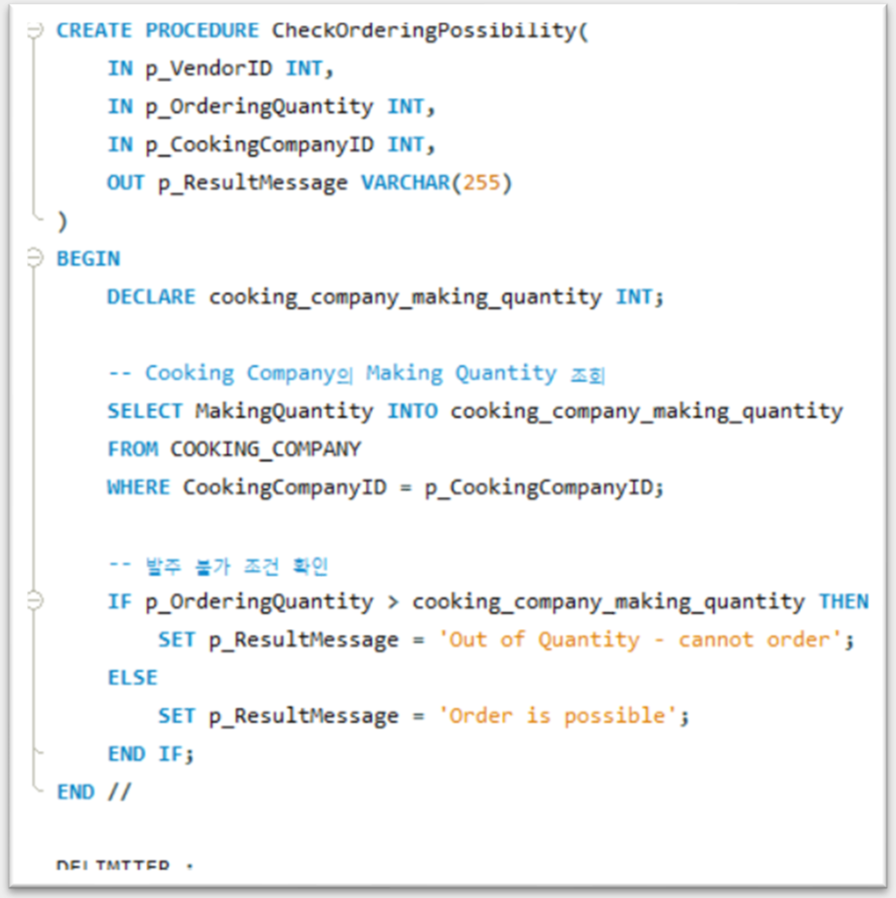
2. Stored Procedure

(1) CheckOrderingPossibility: Vendor가 제조사에 발주 시, 생산량보다 주문량이 많을 시 발주 불가 안내문 생성

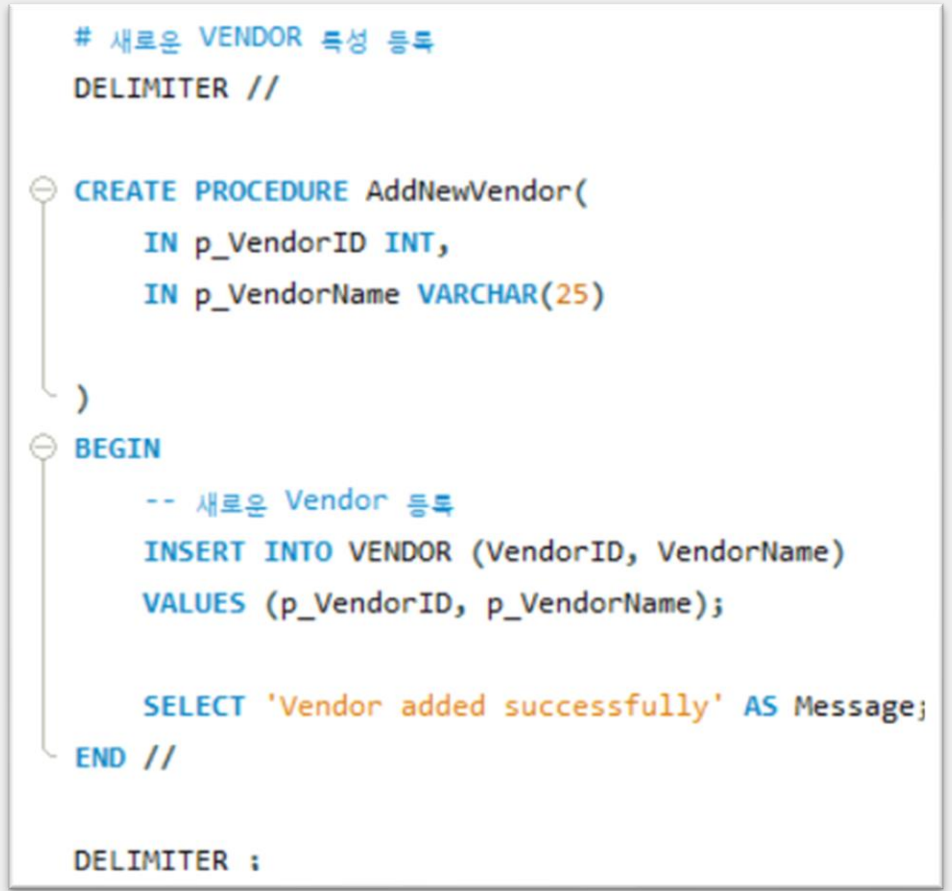
- 생산 가용량을 넘어가는 발주를 방지

(2) AddNewVendor: 새로운 Vendor 및 VendorName을 등록

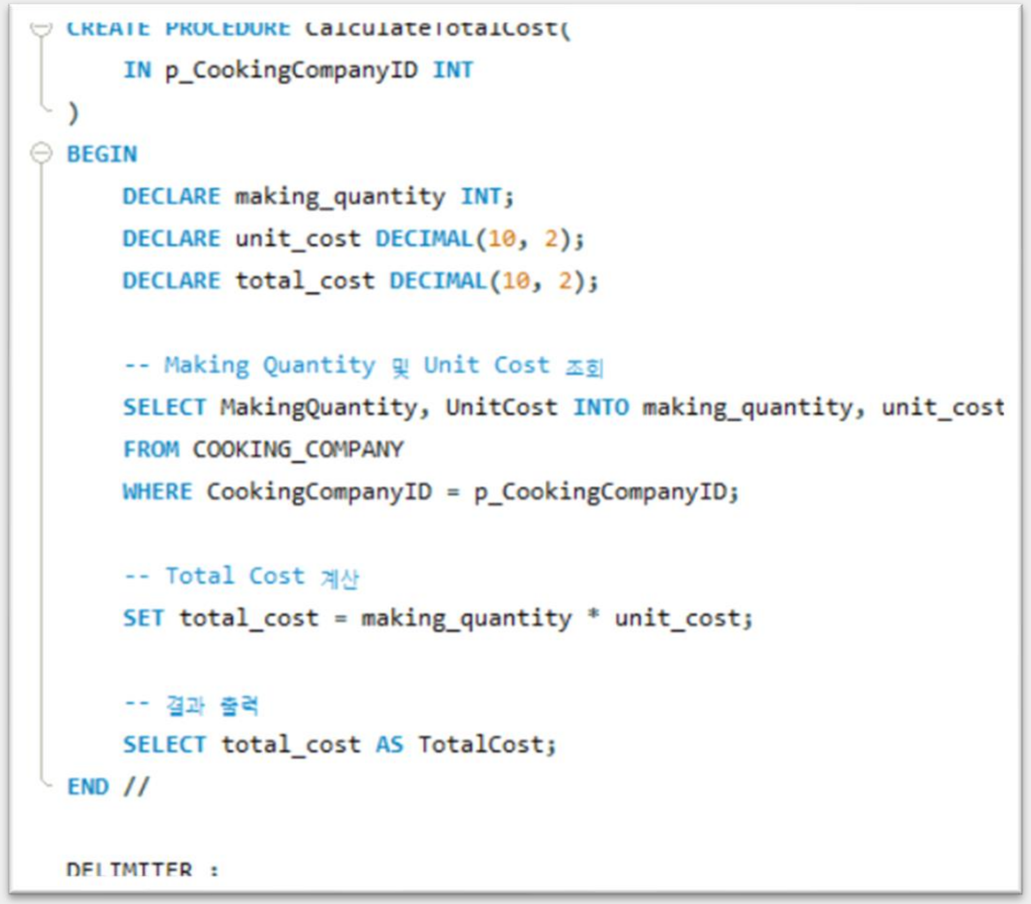
(3) CalculateTotalCost: 제조회사의 총비용 계산



```sql
CREATE PROCEDURE CheckOrderingPossibility(
    IN p_VendorID INT,
    IN p_OrderingQuantity INT,
    IN p_CookingCompanyID INT,
    OUT p_ResultMessage VARCHAR(255)
)
BEGIN
    DECLARE cooking_company_making_quantity INT;

    -- Cooking Company의 Making Quantity 조회
    SELECT MakingQuantity INTO cooking_company_making_quantity
    FROM COOKING_COMPANY
    WHERE CookingCompanyID = p_CookingCompanyID;

    -- 발주 불가 조건 확인
    IF p_OrderingQuantity > cooking_company_making_quantity THEN
        SET p_ResultMessage = 'Out of Quantity - cannot order';
    ELSE
        SET p_ResultMessage = 'Order is possible';
    END IF;
END //

DELIMITER ;
```



```sql
# 새로운 VENDOR 특성 등록
DELIMITER //

CREATE PROCEDURE AddNewVendor(
    IN p_VendorID INT,
    IN p_VendorName VARCHAR(25)

)
BEGIN
    -- 새로운 Vendor 등록
    INSERT INTO VENDOR (VendorID, VendorName)
    VALUES (p_VendorID, p_VendorName);

    SELECT 'Vendor added successfully' AS Message;
END //

DELIMITER ;
```

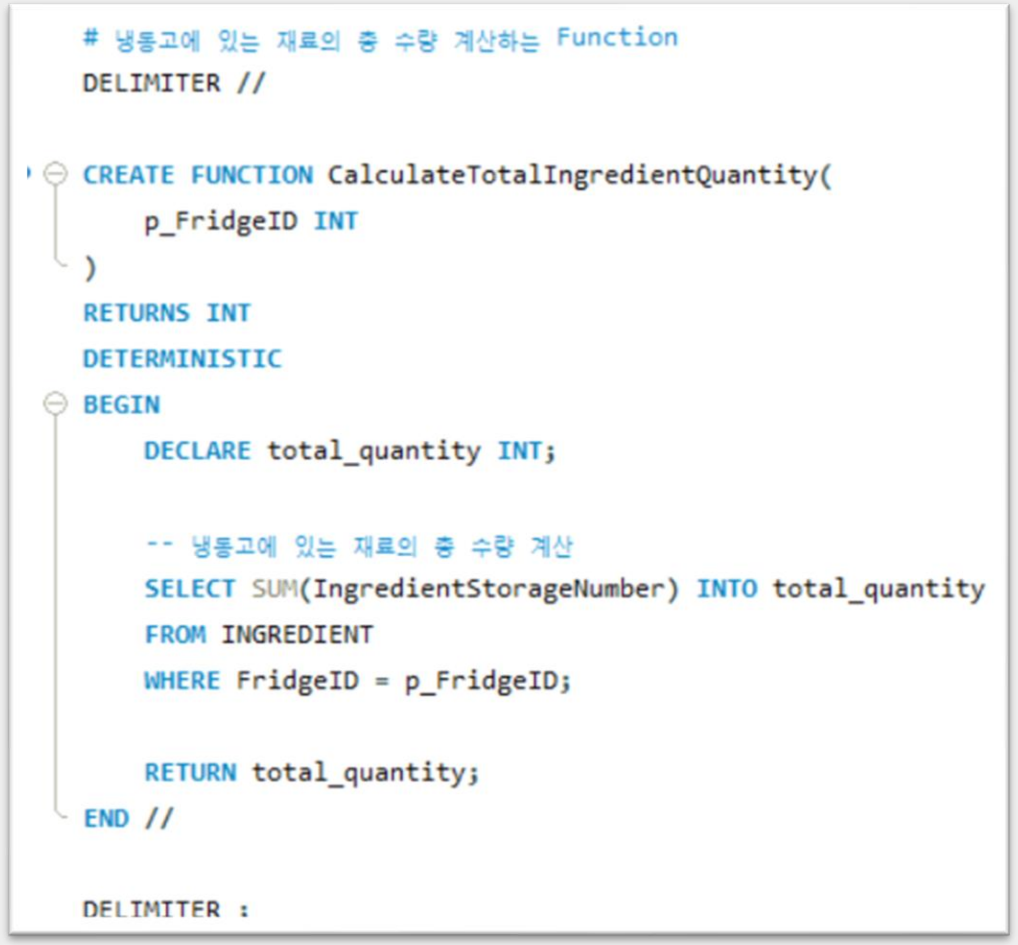


```sql
CREATE PROCEDURE CalculateTotalCost(
    IN p_CookingCompanyID INT
)
BEGIN
    DECLARE making_quantity INT;
    DECLARE unit_cost DECIMAL(10, 2);
    DECLARE total_cost DECIMAL(10, 2);

    -- Making Quantity 및 Unit Cost 조회
    SELECT MakingQuantity, UnitCost INTO making_quantity, unit_cost
    FROM COOKING_COMPANY
    WHERE CookingCompanyID = p_CookingCompanyID;

    -- Total Cost 계산
    SET total_cost = making_quantity * unit_cost;

    -- 결과 출력
    SELECT total_cost AS TotalCost;
END //

DELIMITER ;
```

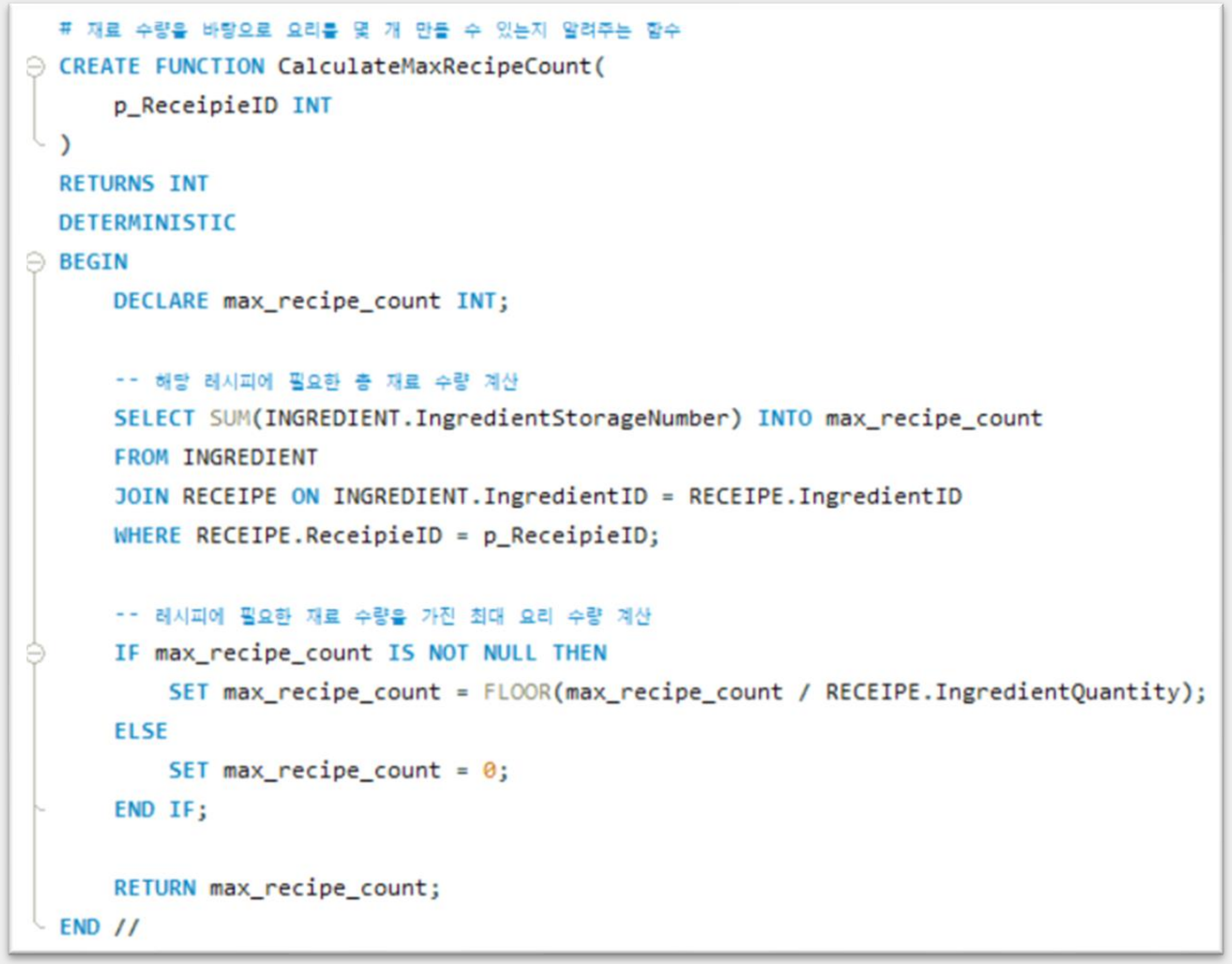
4. SQL/PSM - User Defined Function

3. User - Defined Functions

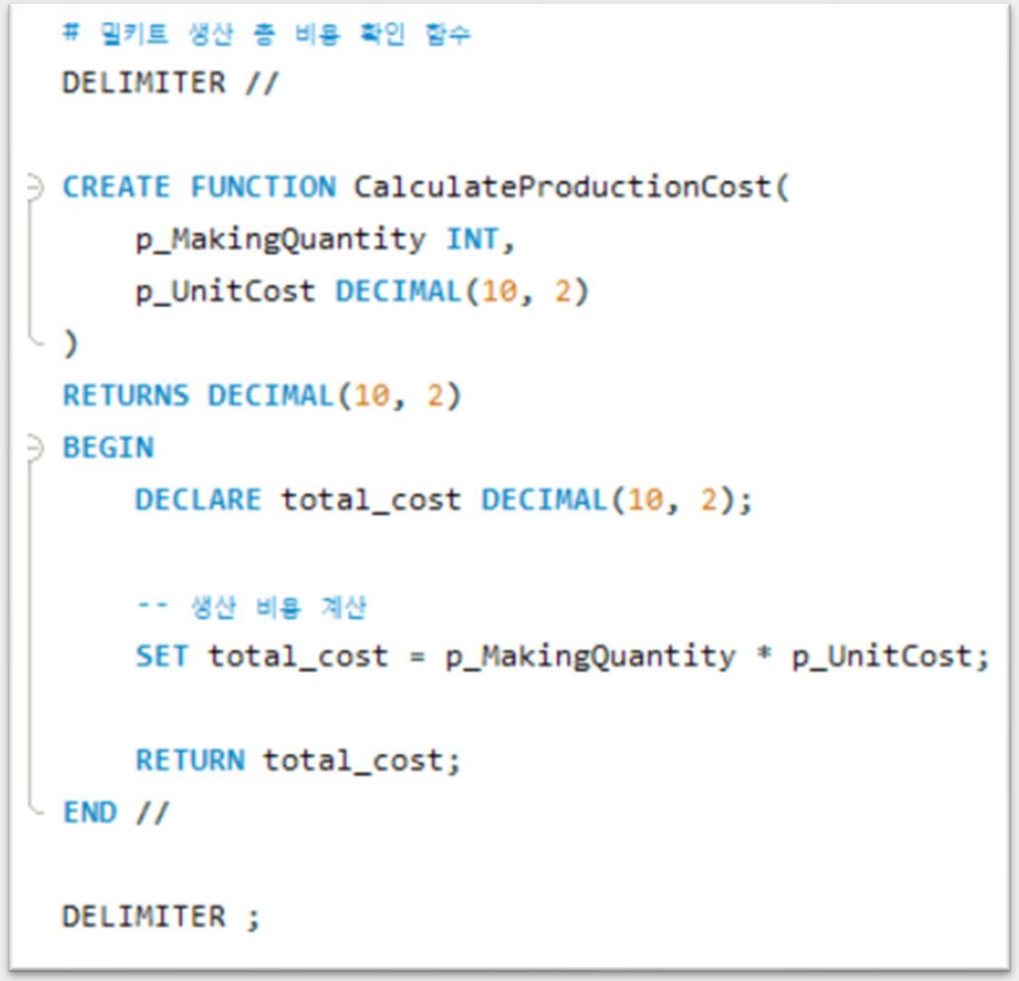
(1) CalculateTotalIngredientQuantity: FRIDGE에 있는 총 재료 수량 계산
(2) CalculateMaxRecipeCount: 재료 수량을 바탕으로 만들 수 있는 요리 개수를 계산
(3) CalaulateProductionCost: 밀키트 생산에 필요한 총 비용 계산



```sql
# 냉동고에 있는 재료의 총 수량 계산하는 Function
DELIMITER //

CREATE FUNCTION CalculateTotalIngredientQuantity(
    p_FridgeID INT
)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE total_quantity INT;

    -- 냉동고에 있는 재료의 총 수량 계산
    SELECT SUM(IngredientStorageNumber) INTO total_quantity
    FROM INGREDIENT
    WHERE FridgeID = p_FridgeID;

    RETURN total_quantity;
END //

DELIMITER ;
```



```sql
# 재료 수량을 바탕으로 요리를 몇 개 만들 수 있는지 알려주는 함수
CREATE FUNCTION CalculateMaxRecipeCount(
    p_ReceipieID INT
)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE max_recipe_count INT;

    -- 해당 레시피에 필요한 총 재료 수량 계산
    SELECT SUM(INGREDIENT.IngredientStorageNumber) INTO max_recipe_count
    FROM INGREDIENT
    JOIN RECEIPE ON INGREDIENT.IngredientID = RECEIPE.IngredientID
    WHERE RECEIPE.ReceipieID = p_ReceipieID;

    -- 레시피에 필요한 재료 수량을 가진 최대 요리 수량 계산
    IF max_recipe_count IS NOT NULL THEN
        SET max_recipe_count = FLOOR(max_recipe_count / RECEIPE.IngredientQuantity);
    ELSE
        SET max_recipe_count = 0;
    END IF;

    RETURN max_recipe_count;
END //
```



```sql
# 밀키트 생산 총 비용 확인 함수
DELIMITER //

CREATE FUNCTION CalculateProductionCost(
    p_MakingQuantity INT,
    p_UnitCost DECIMAL(10, 2)
)
RETURNS DECIMAL(10, 2)
BEGIN
    DECLARE total_cost DECIMAL(10, 2);

    -- 생산 비용 계산
    SET total_cost = p_MakingQuantity * p_UnitCost;

    RETURN total_cost;
END //

DELIMITER ;
```

5. Conclusions

1. Results

- 밀키트 공급망에 관한 DB를 구축하면서, 각 구성원의 최적 의사결정에 도움을 줄 수 있는 PSM, VIEW 등을 제작하였음.
- 제품 총 생산비용, 제조회사의 총 생산비용, 판매점의 총 발주량 확인기능 등을 구현하여 최적 의사결정에 도움을 주었음.
- 이와 같이 구축된 DB를 통해 공급망을 최적화하고, DB의 데이터 분석을 통해 밀키트 산업에 대한 Insight를 얻을 수 있음.

2. Further Improvement

- 처음에는 판매점에서 제품을 구매하는 Customer도 DB에 포함시키려 하였지만, 현실적으로 그것을 구현하는 것이 지나치게 복잡할 것이라고 느꼈음.
  - 유의미한 고객의 데이터는 크기가 지나치게 클 것으로 예상되었고, 그렇다고 데이터의 크기를 줄이면 무의미해질 것으로 예상되었음
  - 따라서 고객에 대하여 유의미한 정보만을 추출하여 데이터를 압축시키거나, 매우 큰 Size의 데이터를 다루는 DBMS를 사용하는 방법 등이 존재함.
- 본 상황(밀키트 산업) 에서 공급망 구성원이 3명으로, 비교적 상황을 간략화 시켜서 나타내었음. 하지만 실제 산업은 그 복잡도가 훨씬 높음.
  - 산업환경에 대한 보다 확실한 이해를 바탕으로, 전체 프로세스에 참여하는 참가자 전원과, 이들의 Cardinality 관계를 확실히 한다.
  - 예를 들어, Fridge에서 Cooking Company로 원자재를 운송 시켜 주는 운송업체 등이 본 DB에 추가될 수 있다.

Thank You